澳大利亚国立大学

计算机学院

JHQ-GPU：面向表示的 GPU 近似最近邻搜索与工作负载级精排预算校准

Rui Luo

导师：Dr Mengxuan Zhang

中文阅读对照稿

第七章为对应英文正文的完整中文译文，章节、公式及图表编号保持一致。其余章节沿用英文总稿的现有骨架。正文、图注与表头已翻译，图内保留原英文标签以便核对。此稿供阅读与审阅。

COMP8800 — Advanced Computing Research Project

2026 年 10 月

Declaration and Contributions

待撰写： Replace this page with the declaration required by the current Canvas instructions. State the work you personally performed, identify inherited JHQ algorithms and code, and distinguish your contributions from those of the supervisor, collaborators, and generative-AI tools. Do not sign an unverified claim of sole authorship or prior non-submission.

Research ownership

待撰写： Describe your contribution to system design, implementation, experimental design, analysis, and writing. Cite the original JHQ paper and identify the relationship to the coauthored ADC manuscript.

Use of generative-AI tools

待撰写： Complete the record in Appendix C. Record actual tools, purposes, relevant prompts or prompt-log locations, and your review of outputs. Match the current course disclosure instructions.

Signature and date

待撰写： Add a signature and date only if the course requires them and after checking the final declaration wording.

Abstract

待撰写： Write this after the results and conclusion. A useful planning length is 250–350 words, subject to Canvas instructions. Cover the problem, your own methods, the strongest verified results, and the principal quality or deployment limitation. Keep experimental versions consistent. Do not describe this template or the writing blueprint in the final abstract.

Acknowledgements

待撰写： Acknowledge actual supervision, collaboration, computing resources, and other assistance. This page is optional unless the course specifies it; remove its input from main.tex if unused.

目录

Declaration and Contributions i

Abstract ii

Acknowledgements iii

Notation and Abbreviations xi

1 Introduction 1

1.1 Context and Practical Objectives 1
1.2 Problem Definition and Scope 1
1.3 Three Linked Technical Challenges 1
1.4 Research Questions and Falsifiable Claims 1
1.5 Contributions and Research Ownership 1
1.6 Thesis Roadmap 2

2 Background and Critical Related Work 3

2.1 Formal ANN Objective and Metrics 3
2.2 IVF, PQ, ADC and Residual Refinement 3
2.3 Inherited JHQ: Rotation, Cartesian Codes and Assumptions 3
2.4 GPU Execution Concepts Used Here 3
2.5 GPU Graph Search: SONG, CAGRA and BANG 4
2.6 Quantised ANN, Fast Tables and Modern Alternatives 4
2.7 Critical Synthesis and Precise Research Gap 4

3 Research Design and Measurement Methodology 5

3.1 Research Questions as Controlled Comparisons 5
3.2 Datasets, Queries, Ground Truth and Splits 5
3.3 Measurement Boundary, Variance and Memory Accounting 5
3.4 Baselines, Tuning Grids and Frontier Construction 5
3.5 Versioned Claim-to-Evidence Ledger 6
3.6 Evidence Strength and Threat Controls 6

4 System Architecture and GPU Index Construction **7**
4.1 End-to-End Build and Search Boundaries 7
4.2 Training, Rotation and Codebook Generation 7
4.3 IVF Assignment, IDs and Physical Memory Layout 7
4.4 Resident Storage, Workspace and Index Lifecycle 7
4.5 Construction Cost and Tested Scale 8
4.6 Index Representation and Safety Contracts 8

5 面向表示的 GPU 搜索执行 **9**
5.1 查询路径与四种不同的排序对象 9
5.2 精确的 Cartesian LUT 分解 10
5.3 打包编码与候选并行的内存访问 12
5.4 通过多 block 扫描实现精确全局主码 Top-c 13
5.5 共享完整 Key 阈值与执行调度 17
5.6 Lane 协作的残差评分与最终 Top-k 20
5.7 作为独立原语的 IVF 探针选择 22

6 Workload-Level Refinement-Budget Calibration **24**
6.1 Why Fixed Refinement Budgets Can Be Wasteful 24
6.2 GT-Free Sampled Output Agreement 24
6.3 Budget Search, Nestedness and Tie Safety 24
6.4 Budget Baselines and Their Questions 24
6.5 Hold-out Risk, Calibration Cost and Payback 25
6.6 Policy Failure Modes and Applicability 25

7 实验评估与研究发现 **26**
7.1 最终实验清单与对照实验组 26
7.2 现代 GPU ANN 的召回率—QPS 前沿 27
7.3 笛卡尔查找表、打包加载与 lane 协作精排 31
7.4 全局 top-c、共享界与探针选择的正确性及消融 33
7.5 留出查询上的预算质量、吞吐量与回本 35
7.6 批大小、扫描组调度与工作负载偏斜 40
7.7 冷构建、常驻显存与已验证规模 42
7.8 数值稳健性、异常与召回率归因 46

8 Discussion, Limitations and Future Research **49**
8.1 Three Cross-Layer Insights 49

8.2 Two Deep Negative-Result Case Studies 49
8.3 Threats to Internal and External Validity 49
8.4 Original Testable Research Directions 49
8.5 Conditional Deployment Recommendation 49

9 Conclusion **50**
9.1 Answers to the Four Research Questions 50
9.2 Closing Principle and Outlook 50

参考文献 **51**

A Reproducibility and Experiment Manifest **53**
A.1 Source and Build Environment 53
A.2 Data and Trained States 54
A.3 Result-to-Figure Mapping 55

B Supplementary Results and Correctness Checks **57**
B.1 Complete Parameter Grids 57
B.2 Correctness and Numerical Checks 57
B.3 Additional Results and Negative Designs 57

C Contributions and Tool-Use Record **58**
C.1 Contribution Record 58
C.2 Generative-AI Assistance 58

D Template Examples — Draft Only **59**
D.1 Citations and Cross-References 59
D.2 A Source-Native Diagram 59
D.3 Tables 59
D.4 Equations and Proofs 59
D.5 Pseudocode and Code 60

插图目录

5.1 查询执行及其独立选择边界。路由选择列表，主码筛选选择向量候选，最终选择对精排后的保留候选排序。原空间 ground truth 用于评价，不作为这些搜索决策的输入。 10
5.2 打包后的候选并行访问，图中仅画出部分 warp lane。固定字组 j 时，连续候选位置访问相邻四字节 word。每个 word 包含同一候选的四个子空间编码，解包保持编码值，随后再进行 LUT 拆分。此图说明地址关系，不表示实测内存事务数量。 13
5.3 局部覆盖与截断式两两合并，示例中 $c = 3$、$G = 4$。条目是 (D_{P}, p) key，属于示意而非测量数据。粗体条目是最终全局优胜候选。短组仅贡献有效前缀，空组不贡献条目。每次合并使用同一 key 顺序，并保留至多 c 项。 15
5.4 单个扫描组的共享完整 key 过滤。候选的完整主码分数在接纳之前计算。有效的 c 候选见证集合可以收紧查询级共享界；陈旧但完整的界只会削弱过滤。最终前缀裁剪减少合并的有效输入，但不改变已分配工作区的容量。完整 key 读取这一前提的实现边界在正文中另行说明。 18
5.5 十六 lane 残差精排。一个 warp 处理两个候选；候选内的 lane a 读取残差 word $a, a + 16, \ldots$。残差编码为八比特时，每个 32-bit word 解码四个坐标。子组归约合并部分和，再加上保留的主码分数和修正项。即使子组内 word 访问连续，候选物理位置仍可能分散。 . . 21

7.1 四个 E1 数据集上实测的主机到主机召回率—QPS 前沿。图中从召回率 0.90 开始显示，并在 0.97 处标出竖线；构造前沿时纳入了包括 CAGRA int8 在内的全部基线组。连线连接实测点，吞吐量轴为对数坐标。 29
7.2 固定 $\alpha = 100$ 的笛卡尔查找表分组实验（v71）。组数表示 g_{LUT}，不是并行扫描组数。曲线为各数据集相对单组 QPS 比值的中位数，阴影表示范围。 31

7.3 打包访存与精排对照。左：各查找表分组和探针数下，六个数据集打包加载 QPS 增益的中位数，阴影为最小值至最大值。右：128 探针及各数据集普通预算下，仅改变加载和使用 16 个 lane 的实测端到端收益。两个子图来自不同实验组。 32
7.4 不同样本大小下的留出风险。纵轴为相对最佳被测预算，留出召回率损失超过 10^{-3} 的校准抽样百分比。由于式 (7.1) 始终保持阈值一，增加 S 也收紧了允许的槽位不一致比例。 37
7.5 相对固定预算 100 和 64 的校准回本估计。实心圆表示 128 探针，空心三角表示 512 探针；顶行点没有吞吐量回本。估计对所选配置平均 QPS 取倒数，使用 500 条查询，Stella 为 499。 40
7.6 执行区间。左：共享界固定组数与调度执行，相对 lane 精排单线程块基线的吞吐量比，较小批次有 13 个单元，批大小 1,000 有 11 个。右：arXiv 预算 100 和 200 在批大小 8 与 1,000 下的配对扫描、合并剖析；斜线表示合并时间。右侧来自独立界扫描实验，用于解释一项成本，不代表调度器全部收益。 41
7.7 包括 CAGRA int8 的已记录构建成本。JHQ 为一次冷训练加添加记录；其他柱形为成功测量的索引配置之间的中位数，误差线表示最小值至最大值。Int8 包含数据集量化；RaBitQ 不包含所需的序列化往返。大数据集缺失的柱形表示未尝试的配置，不代表实测构建失败。纵轴为对数坐标。 43
7.8 两组查找表、128 探针、$\alpha = 100$ 下的常驻显存分解。组件大小由模型计算，标记为实测已用总量，斜线区域为未归因部分。总量包含上下文与工作区，未测量构建峰值。 44
7.9 大数据集上的主机到主机前沿，包含 CAGRA int8 成功的隔离重跑。BGE 使用 1,000 条查询，Stella 使用 999 条。曲线端点是已测试网格的终点，不是方法的内在召回率上限。 45

D.1 Formatting example: a schematic query path. Replace or extend it with the verified execution boundaries of the implementation. . . . 59

表格目录

5.1 八比特、八坐标主码子空间的 Cartesian 表权衡。合并加法为组合查表结果所需的代数最小次数，不含子空间之间的累加，也不是实测指令数。 11
5.2 各选择阶段的对象、并列规则和 reference checks 不同。一个阶段的证据不能建立另一阶段的正确性。 23

7.1 实验与证据分组。数量表示测量配置或检查次数，不代表独立索引构建次数。所有实验均使用 RTX 5090，但计时协议有所不同。 . . 27
7.2 召回率 0.97 下的比较。每组比值依次为“插值 / 实测阈值”；大于一表示 JHQ 更快。JHQ kQPS 为插值估计。arXiv 与 RaBitQ 的微小差异随估计方法而改变方向。 29
7.3 128 探针下的精排结果。除最后一列为剖析得到的精排内核加速比外，其余比值均为相对 L0 的配对端到端收益。不能将内核加速比与应用整体加速比混用。 32
7.4 精确 FP32 探针选择改动的 v69/v65 端到端吞吐量比。每个工作负载—批大小比较中，每个二进制程序共计时十轮，采用交替运行。大于一表示 v69 更快。 35
7.5 $S = 128$ 时的留出质量。规则的 α 是 64 次抽样的中位数，oracle 的 α 使用留出标签。$\overline{R}_{\mathrm{rule}}$ 是重构的平均召回率。“超标”统计相对最佳被测留出召回率损失 $> 10^{-3}$ 的抽样次数；L_{95} 是日志采用较低次序统计量得到的 95 分位损失估计。R_{100} 是固定预算召回率。 . . . 37
7.6 十二个留出单元上的固定预算对照。达标表示按打印精度，距最佳被测召回率不超过 10^{-3}。最大召回率损失使用全部单元，QPS 差额只统计该预算达标的单元。横线表示没有达标单元。 38
7.7 校准经济性，$S = 128$。速率单位为 kQPS，规则速率为所选配置平均 QPS。比值分别相对固定 100 与固定 64。最后一列回本相对固定 100，使用 500 条查询，Stella 为 499；“不回本”表示没有吞吐量回本。 39

7.8 集成执行相对 v61 L16 的收益。每项先计算单元内五轮配对加速比的中位数，再跨单元取几何平均。“留出配置” 排除了设置规则时使用的四个配置，指的是未用于设定规则的配置，不是预算实验中的留出查询划分。 42
7.9 JHQ 冷构建与单独测量的常驻显存。训练和添加来自每个数据集的一次已记录冷构建；显存来自 v71 两组、128 探针、$\alpha = 100$ 的运行，包含上下文与工作区。“残差占比” 表示模型计算的残差码占实测已用总量的比例，而不是每次查询读取的比例。 43
7.10 E1 分配范围。JHQ 为进程总量减去 507.2 MiB 上下文估计，包含查询工作区；CAGRA fp32 为构建时测量的索引分配。比值范围组合每个数据集的极值。 45
7.11 本章证据支持的四个研究问题答案。 48

A.1 Component and run provenance. “Report” distinguishes a version stated in an experiment report from a raw run-header commit. The exact E1 build state is not inferred from a baseline’s metadata. . . 54
A.2 Chapter 7 result-to-figure mapping. Paths below are relative to `JHQ_GPU`; detailed file identities are in the hash ledger. 55

D.1 Example manifest fields. This table contains no experimental results. 60
D.2 Pseudocode layout example. 60

Notation and Abbreviations

Symbol	Meaning
N, d	Number of database vectors and original dimension
M, D_s	Number of primary subspaces and dimensions per subspace
k	Number of returned neighbours
n_c, c	Routed candidates and primary survivors receiving refinement
α	Refinement multiplier, with $c = \max\{k, \lceil \alpha k \rceil\}$
S, ϵ	Calibration sample size and allowed disagreeing result slots
g_{LUT}	Number of groups in a factorised lookup table
g_{scan}	Number of scan groups per query
ℓ_{ref}	Number of cooperating lanes per refined candidate
D_{raw}	Squared distance between original vectors
D_{P}	Distance under the primary reconstruction
D_{H}	Refined ranking score: reconstructed squared distance plus a query-only constant

Abbreviation	Meaning
ANN	Approximate nearest neighbour
ADC	Asymmetric distance computation (distinct from the ADC conference)
IVF	Inverted file
JHQ	Johnson–Lindenstrauss enhanced hierarchical quantisation
LUT	Lookup table
PQ	Product quantisation
QPS	Queries per second
GT	Ground truth

待撰写： Check every definition against the final implementation and remove symbols that are not used. Keep code parameter names distinct from mathematical notation, especially the multiple uses of G and B.

第 1 章

Introduction

待撰写： Establish the research problem, testable questions, scope, and personal contributions. Keep results consistent with the final evidence manifest.

1.1 Context and Practical Objectives

待撰写： Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

1.2 Problem Definition and Scope

待撰写： Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

1.3 Three Linked Technical Challenges

待撰写： Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

1.4 Research Questions and Falsifiable Claims

待撰写： Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

1.5 Contributions and Research Ownership

待撰写： Identify personal work, collaborator roles, inherited algorithms and code, the ADC manuscript relationship, and actual tool assistance.

1.6 Thesis Roadmap

待撰写： Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

第 2 章

Background and Critical Related Work

待撰写： Explain only background used by the thesis. Compare alternatives critically and distinguish inherited JHQ properties from this work.

2.1 Formal ANN Objective and Metrics

待撰写： Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

2.2 IVF, PQ, ADC and Residual Refinement

待撰写： Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

2.3 Inherited JHQ: Rotation, Cartesian Codes and Assumptions

待撰写： State that the orthogonal transform is square. Define the primary codebook using its actual formula, residual relative to primary reconstruction, and valid parameters.

2.4 GPU Execution Concepts Used Here

待撰写： Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

2.5 GPU Graph Search: SONG, CAGRA and BANG

待撰写： Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

2.6 Quantised ANN, Fast Tables and Modern Alternatives

待撰写： Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

2.7 Critical Synthesis and Precise Research Gap

待撰写： Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

第 3 章

Research Design and Measurement Methodology

待撰写： Define comparison protocols once. Separate exploratory findings from questions specified before measurement; identify the actual evidence for each claim.

3.1 Research Questions as Controlled Comparisons

待撰写： Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

3.2 Datasets, Queries, Ground Truth and Splits

待撰写： Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

3.3 Measurement Boundary, Variance and Memory Accounting

待撰写： Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

3.4 Baselines, Tuning Grids and Frontier Construction

待撰写： Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

3.5 Versioned Claim-to-Evidence Ledger

待撰写： Record commit, binary, compiler options, index/cache identity, query split, raw result, metric, and figure-generation script for every main claim.

3.6 Evidence Strength and Threat Controls

待撰写： Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

第 4 章

System Architecture and GPU Index Construction

待撰写： Describe build-time dataflow, CPU/GPU responsibilities, physical storage, temporary memory, and representation invariants.

4.1 End-to-End Build and Search Boundaries

待撰写： Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

4.2 Training, Rotation and Codebook Generation

待撰写： Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

4.3 IVF Assignment, IDs and Physical Memory Layout

待撰写： Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

4.4 Resident Storage, Workspace and Index Lifecycle

待撰写： Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

4.5 Construction Cost and Tested Scale

待撰写： Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

4.6 Index Representation and Safety Contracts

待撰写： Distinguish representation checks, algorithmic selection correctness, numerical agreement, and memory-safety checks; do not use recall equality as proof.

第 5 章

面向表示的 GPU 搜索执行

本章回答 RQ2：如何使 JHQ 的压缩表示支持正确且高效的 GPU 执行。变换、Cartesian 主码码本及残差层次继承自 JHQ[4]；这里关注它们的执行方式。论证沿查询路径，从路由到主码选择，再到残差评分。对每种机制，分别讨论数学目标、实现条件和资源权衡，对应测量见第7章。工作负载级精排预算选择则在第6章单独展开。

5.1 查询路径与四种不同的排序对象

对固定索引，查询先变换为 $q' = \Pi q$，再路由到选中的倒排列表，使用主码筛选，仅对保留候选做精排。最终选择器返回 k 个数据库 ID 及计算所得分数。图 5.1区分运行时的三个选择决策；ground-truth 排序属于外部评价对象。

需要分清四种量：

$$\begin{aligned} s_i(q) &= \|\mu_i\|_2^2 - 2\langle q', \mu_i\rangle, && \text{centroid routing score,} \\ D_{\mathrm{P}}(q,x) &= \|q' - \widehat{p}(x)\|_2^2, && \text{primary score,} \\ D_{\mathrm{H}}(q,x) &= \|q' - \widehat{p}(x) - \widehat{r}(x)\|_2^2 + \|q'\|_2^2, && \text{refined ranking score,} \\ D_{\mathrm{raw}}(q,x) &= \|q - x\|_2^2, && \text{raw-space distance.} \end{aligned} \tag{5.1}$$

其中 μ_i 是变换坐标中的 IVF 中心，$\widehat{p}(x)$ 为主码重建，$\widehat{r}(x)$ 为解码后的残差。运行时精排分数包含一个仅与查询有关的常数；第5.6节依据实际修正项推导这一点。对固定查询，它保持重建距离的排序，但精排分数和主码分数都不等于原始向量距离。

路由计算中心分数并选择 n_{probe} 个列表（第5.7节）。拼接这些列表得到大小为 n_c 的已访问集合 U。精确探针原语只是按已计算路由分数选择所需中心，不证明 U 包含全部原空间近邻。前缀偏移还给出扫描与调度所需的实际候选数。

主码筛选构造每查询 LUT，读取已访问向量的主码，再按 D_{P} 保留至多 $c = \max\{k, \lceil \alpha k \rceil\}$ 个候选。固定表分组时，分数计算成本为 $O(n_c M)$，此外还有局部选择和合并。Sections 5.2与5.3讨论表与编码访问，Sections 5.4与5.5建立选择不变量，并分析增加并行度的成本。

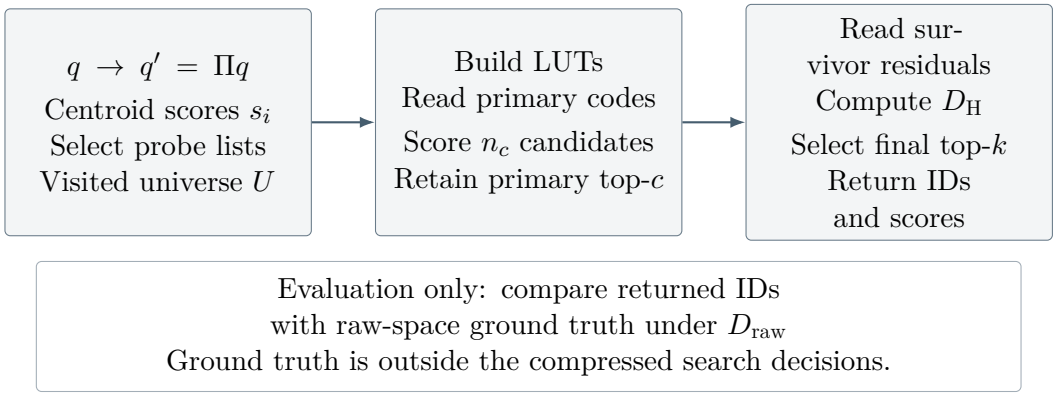


图 5.1: 查询执行及其独立选择边界。路由选择列表，主码筛选选择向量候选，最终选择对精排后的保留候选排序。原空间 ground truth 用于评价，不作为这些搜索决策的输入。

残差评分只对有效保留候选读取残差码，进行 $O(cd)$ 坐标工作，并向最终选择器提供 D_{H}（第5.6节）。这一路径使用压缩重建替代搜索时的原始向量读取。实现中的最终 repeated-minimum 选择器为每个输出扫描 c 个槽位，共输出 k 个，另有 block 归约工作。

确定性排序 key 为各类分数附加并列规则：中心选择使用列表 ID，完整 key 主码选择使用物理行位置，最终选择使用数据库 ID。旧的查询级主码路径具有不同的截断并列规则，见Sections 5.4与5.5的限定。因此，分数保持、保留集合一致、有序输出相等及原空间 recall 是不同主张。第七章在各自边界上检查它们，而不把单个 recall 不变当作整条流水线的证明。

5.2 精确的 Cartesian LUT 分解

广泛扫描通过非对称距离计算评估主码重建：查询保持 FP32，存储编码用于选择重建坐标 [4, 5]。对变换后的查询 q'，主码子空间 m 的普通查找表为

$$T_m[z] = \|q'_m - C_m[z]\|_2^2, \qquad D_{\mathrm{P}}(q, x) = \sum_{m=0}^{M-1} T_m[z_m(x)]. \tag{5.2}$$

八比特码要求每个子空间存储 256 项。这里的优化机会来自码本内部的 Cartesian 结构，而不只是向量被划分成 PQ 子空间。

5.2.1 在不改变量化器的情况下分解

对被测 $D_s = 8$ Cartesian 码本，高半字节 h 选择前四个重建坐标，低半字节 l 选择后四个。因此 $C_m[16h + l] = (A_m[h], B_m[l])$，两个坐标组之间不存在交叉依赖。

表 5.1: 八比特、八坐标主码子空间的 Cartesian 表权衡。合并加法为组合查表结果所需的代数最小次数，不含子空间之间的累加，也不是实测指令数。

g_{LUT}	表项	FP32 字节	查表次数	合并加法	建表坐标项
1	256	1,024	1	0	2,048
2	32	128	2	1	128
4	16	64	4	3	32
8	16	64	8	7	16

定义

$$T_m^{\mathrm{hi}}[h] = \|q'_{m,\mathrm{hi}} - A_m[h]\|_2^2, \qquad T_m^{\mathrm{lo}}[l] = \|q'_{m,\mathrm{lo}} - B_m[l]\|_2^2. \tag{5.3}$$

平方 L2 对互不相交坐标可加，得到

$$T_m[z] = T_m^{\mathrm{hi}}[z \gg 4] + T_m^{\mathrm{lo}}[z \,\&\, 15]. \tag{5.4}$$

只需存储 32 项，而不是 256 项。重建表示和实数运算下的分数保持不变，无需重训练编码、增加量化或改变候选集合。

建表程序从现有码本读取代表码字 $16h$ 和 l，只计算相应坐标片段；另一片段的零编码位与此计算无关。这要求所有共享同一半字节的码字也共享对应坐标片段。`verify_split_lut` 将码字片段与代表码字比较，拒绝不兼容码本。任意学习得到的 256 中心 PQ 子码本不一定满足该条件；仅将编码标签拆成半字节，不会自动分解中心坐标。

数学恒等式不保证 FP32 逐位相等。八个坐标的连续累加，与两个四坐标和再相加，浮点结合顺序不同。因此实现保留 FP32 表，并单独检查数值和排序影响。将表项改成 FP16 会增加另一种舍入，是不同的设计选择。

5.2.2 分组、建表工作量与资源成本

v71 消融将分解推广到 $g = g_{\mathrm{LUT}} \in \{1, 2, 4, 8\}$ 个连续坐标组，每组位字段与 Cartesian 数字对应。当编码为八比特、$D_s = 8$ 时，每组有 $2^{8/g}$ 项；总表项和每查询 FP32 存储为

$$E(g) = g2^{8/g}, \qquad W_{\mathrm{LUT}}(g) = 4ME(g) \text{ bytes}. \tag{5.5}$$

独立构造表项时，每个子空间计算 $D_s 2^{8/g}$ 个坐标贡献。相比独立计算完整表的每一项，两组设计将表存储减少八倍、坐标工作量减少十六倍。这是表级成本比，不是应用性能加速预测。

表先在全局工作区构造。如果表和该 kernel 的候选缓冲区能放入设备配置的容量，扫描 block 就合作将表搬到共享内存，再通过 block 屏障完成搬运；否则读取全局表。共享内存占用必须包含选择工作区，不能仅由 W_{LUT} 判断驻留。多个分组扫描 block 还可能各自搬运同一查询的表，因此这一设计与第5.5节的选择有关。

当 $M = 96$ 和 384 时，每查询完整 FP32 表分别需要 96 和 384 KiB，两组表仅需 12 和 48 KiB。较小占用提供更多放置机会，但四组和八组即使均只存储每子空间 16 项，也要做更多查表和结果组合。索引提取、依赖查表、同步和驻留均参与最终权衡。不能仅凭表大小判断最快设计，或确定唯一的 cache / bank-conflict 原因。

小表执行在 PQ Fast Scan 和 Quicker ADC 中已有重要先例 [1, 2]。这里将精确坐标分解应用于 JHQ 现有 Cartesian 码本，并测量 GPU 分组权衡，不宣称首次提出小 LUT。第7.3节报告受控 v71 扫描：两组在 24 个单元中领先 22 个，另外两组差距均不足 1%。结果支持选定运行点，而非普遍最优结论。

5.3 打包编码与候选并行的内存访问

LUT 分解减少每查询的查找表；编码布局决定各 lane 如何取得候选对应的表索引。这是两项独立优化。扫描将相邻 lane 分配给相邻候选出现位置，因此广泛扫描适合按子空间跨候选存储编码，而不是将每个候选的完整主码连续存放。

5.3.1 地址映射与四字节打包

令 o_ℓ 为倒排列表 ℓ 的起点，r 为列表内行号，$p = o_\ell + r$ 为物理位置。若 $P[m, p]$ 表示主码子空间 m 的八比特编码，逐字节扫描副本将其存于字节偏移 $mN + p$，而候选优先布局使用 $pM + m$。当各 lane 访问同一列表内连续位置时，前者使固定 m 的访问相邻。跨列表边界时，位置可能跳跃，因此不保证每个 warp 都对应一次连续事务。

打包表示将同一候选的四个子空间编码组成一个无符号字：

$$W[j, p] = \sum_{t=0}^{3} 2^{8t} P[4j + t, p], \qquad \mathrm{offset}_{\mathrm{bytes}}(W[j, p]) = 4(jN + p). \tag{5.6}$$

通过 $(W[j, p] \gg (8t)) \,\&\, 255$ 精确恢复四个编码。每个编码再提供式 (5.4) 中的高、低半字节索引。打包四个子空间，与拆分一个子空间的八比特标签不同，也不等于打包四个候选的编码。

建库路径明确将 $[M, N]$ 字节数组重新打包为 $[M/4, N]$ word 数组。仅把原字节数组重新解释为 `uint32_t`，会合并相邻候选的字节，而不是需要的子空间。实

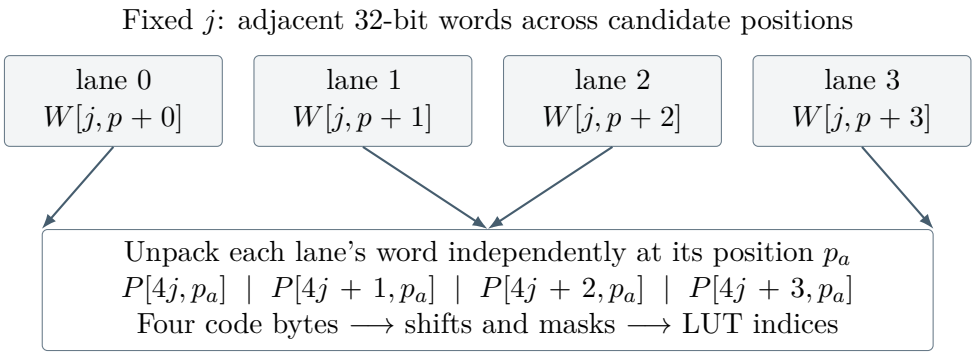


图 5.2: 打包后的候选并行访问，图中仅画出部分 warp lane。固定字组 j 时，连续候选位置访问相邻四字节 word。每个 word 包含同一候选的四个子空间编码，解包保持编码值，随后再进行 LUT 拆分。此图说明地址关系，不表示实测内存事务数量。

现要求 M 能被四整除，并拒绝不完整的最后一个 word。重排在建库时读写一次 NM 编码字节，旧存储和新存储短暂共存；它位于搜索计时边界之外。

5.3.2 打包访问省下什么、保持什么

在源码层面，一个候选用一次 word load 取得四个编码，替代四次 byte load，然后在寄存器中移位和掩码解包。同一列表连续区间内，32 个 lane 请求逻辑上共 128 字节的 word 窗口。实际事务依赖对齐、活跃 lane 和内存层次。源码 load 操作减少四倍，不代表事务数或耗时也减少四倍。

逻辑载荷仍是每候选 M 字节；重排没有进一步压缩索引。逐字节和 word 两种方案本来就按子空间或子空间组组织扫描副本。因此，历史开关对照测量的是打包布局及其加载 / 解包路径的净影响，不是从候选优先布局进行物理转置的独立贡献，也不能唯一分离指令数、寻址和 cache 的效果。

第7.3节报告 48 个匹配的 byte / word 配置，吞吐比为 1.012–1.658×。每个 arm 只有一个汇总速率，其波动证据弱于较新的重复 LUT 实验。整数表示恒等式是精确的，而最多 10^{-4} 的实测 recall 差异属于整条被测执行路径。方法主张是在保持编码值的情况下减少加载 / 寻址操作，性能解释则受实验测量范围限制。

5.4 通过多 block 扫描实现精确全局主码 Top-c

主码筛选既需要暴露足够的 GPU 并行工作，又不能丢掉本应进入残差精排的候选。以查询为单位的扫描为每条查询启动一个 block；当 batch 很小时，即使每条查询访问大量向量，也只有少量 block 可供执行。GPU 上的选择操作本身就是一个重要的设计问题 [6]。v62 引入的分组扫描将已访问候选划分给多个 block，再合

并各组保留的候选。它保证的是基于已计算主码分数的精确选择；IVF 路由和量化表示仍然具有近似性。

5.4.1 候选划分与确定性顺序

对固定查询，按探针顺序拼接选中的倒排列表，得到 n_c 个候选出现位置。假设每个被访问数据库行只出现一次，搜索期间索引保持不变。对 $G = g_{\text{scan}} \in \{1, 2, 4, 8\}$，block (q, g) 负责区间

$$U_g = \left[\left\lfloor \frac{g n_c}{G} \right\rfloor, \left\lfloor \frac{(g+1) n_c}{G} \right\rfloor \right), \qquad g = 0, \ldots, G-1. \tag{5.7}$$

这里的区间表示相应候选，而不是它们的数据库 ID。设备端依据路由得到的候选数计算区间端点。各组候选数最多相差一个；一个较长的列表可以跨越组边界。这种划分平衡的是候选数量，而平均分配列表数量不一定能达到同样效果。每个线程通过探针前缀偏移找到自己的第一个列表，在扫描区间时向前移动列表游标。此阶段不读取残差码。

选择使用的 key 为

$$K_q(x) = \big(D_{\mathrm{P}}(q, x), p(x)\big), \qquad (a, p) \prec (b, r) \iff a < b \text{ or } (a = b \text{ and } p < r), \tag{5.8}$$

其中 $p(x)$ 是固定倒排存储中的物理行位置。它在该查询的候选中唯一，不依赖原子追加的先后顺序，也不是最终结果处理并列时使用的数据库 ID。当分数不含 NaN 时，这一定义为不同候选建立全序。局部比较和合并决策都使用相同顺序。要使选择不受划分方式影响，还要求各条比较路径为同一候选计算出相同的分数；排序规则本身不能消除浮点评分差异。

5.4.2 精确局部保留与全局覆盖

用 $\mathrm{Top}_c(V)$ 表示将 V 按 K_q 排序后最前面的 $\min(c, |V|)$ 个候选，其中 $c = \max\{k, \lceil \alpha k \rceil\}$。每个独立组保留完整的局部 top-$c$，而不是分得一个 c/G 配额。这一区别不可省略：当 $c = 4$、组数为二时，四个全局优胜候选可能全在同一组。如果每组只保留两个，即使总容量为四，仍会丢掉全局优胜候选。

命题 5.1 (局部覆盖与截断合并)**.** *给定互不相交的组 $U_0, \ldots, U_{G-1}$、固定候选分数及共同全序，有*

$$\mathrm{Top}_c\left(\bigcup_g U_g \right) = \mathrm{Top}_c\left(\bigcup_g \mathrm{Top}_c(U_g) \right). \tag{5.9}$$

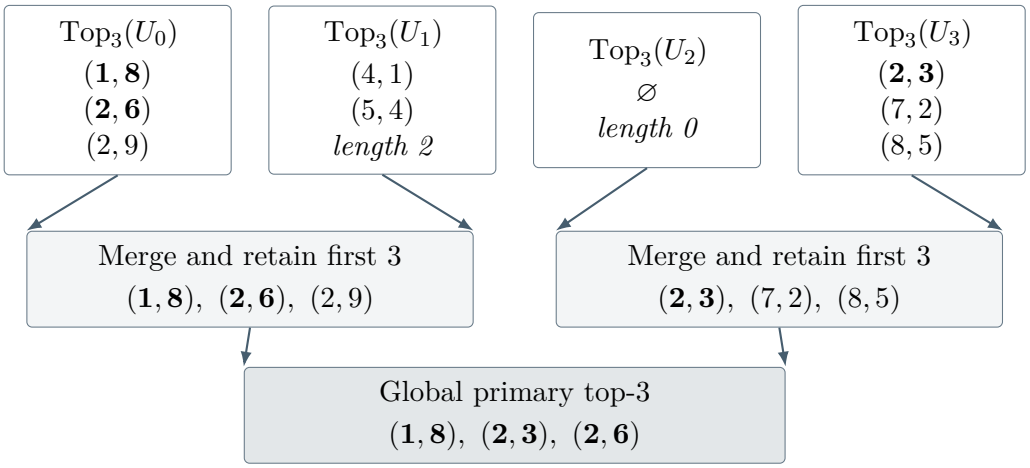


图 5.3: 局部覆盖与截断式两两合并，示例中 $c = 3$、$G = 4$。条目是 (D_{P}, p) key，属于示意而非测量数据。粗体条目是最终全局优胜候选。短组仅贡献有效前缀，空组不贡献条目。每次合并使用同一 key 顺序，并保留至多 c 项。

对有序列表两两合并，并在每次合并后截取前 c 项，保持这一结果；候选总数不足 c 时也成立。

证明. 设 x 同时属于全局 top-c 和组 U_g。全局排在 x 之前的候选少于 c 个，因此组 U_g 内排在它之前的候选也少于 c 个。于是 $x \in \mathrm{Top}_c(U_g)$，所有局部结果的并集包含每个全局优胜候选。再从这个并集中取前 c，就精确恢复同一顺序下的全局结果。如果整个候选集合不足 c，所有成员均被保留。

对于合并，两个有序列表的前 c 项就是二者并集的 top-c。被截去的候选在该并集中已经有至少 c 个前驱，不可能再进入任何更大并集的 top-c。对合并树的各轮归纳即可得到结论。 □

CUDA 实现在共享内存中维护容量为 C 的 key 对缓冲区。扫描 block 含 T 个线程，C 为二的幂且满足 $C \geq c + T$。压缩操作通过 bitonic 网络排序，保留至多 c 个有效条目；当有效条目达到 c 个时，将最后一个保留 key 记为局部阈值。在此之前，阈值是排在所有有效 key 之后的哨兵。两次压缩之间，仅追加排在阈值之前的候选。即使后续候选继续到来，阈值对应的 c 个见证候选仍使它成为已扫描前缀中最优 c 项的安全上界。

每个 chunk 最多追加 T 项。处理完一个 chunk 后，如果下一个 chunk 可能使缓冲区越界，就先压缩。压缩后至多剩下 c 项，$C \geq c + T$ 保证容得下下一个 chunk；不压缩时，显式容量检查提供同样保证。该不变量防止溢出，也不限制一个线程可以遇到多少个有竞争力的候选。固定容量的逐线程保留缓冲区并不具有相同的覆盖保证。

另一种方案是先将全部 n_c 个分数 / 位置对写出，再调用独立选择器。这保持相同目标，但增加一个 $O(n_c)$ 中间数组及其写入。这里将打包主码的评分直接送

入有界局部缓冲区，以重复局部排序换取 $O(Gc)$ 规模的中间输出。基于寄存器的选择也是已有的 GPU 设计 [6]；本轮实验评估选定的扫描变体，并未证明其优于所有库选择器。

5.4.3 合并实现、存储与边界情况

当 $G > 1$ 时，每条查询由一个 256 线程的 block 执行 $\log_2 G$ 轮两两合并。有效长度为 a, b 的一对列表输出 $\min(c, a + b)$ 个 key。Merge Path 将输出划分为连续的线程片段：先二分查找每个片段对应的输入偏移，再由该线程顺序合并自己的片段 [3]。所有比较都遵循式 (5.8)，各轮之间通过 block 屏障同步。图 5.3 展示了截断不变量。除划分查找与同步工作外，合并树全部内部节点合计最多输出 $c(G - 1)$ 个 key 对。

两个全局分数 / 位置工作区的容量分别为 $[b_{\text{cap}}, G, c]$ 和 $[b_{\text{cap}}, G/2, c]$，在各轮间交替使用。分数和位置各占四字节，因此二者的总载荷为

$$W_{\text{pairs}} = 8 b_{\text{cap}} c \left(G + \frac{G}{2} \right) \quad \text{bytes}, \qquad G > 1. \tag{5.10}$$

各组计数还需要 $4 b_{\text{cap}} G$ 字节；配置共享界存储时另需 $8 b_{\text{cap}}$ 字节。这些空间位于最终保留数组及其他搜索工作区之外。分配容量在 graph capture 之前增长，之后保留复用，因此以后选择较小的 G 不一定释放预留显存。合并只在共享内存中保存列表长度，而不存放全部 Gc 个 key。

局部有效前缀由计数界定，而不是仅依靠哨兵分数。压缩时，排序缓冲区填充 $(+\infty, \text{INT_MAX})$；有效位置必须小于这个保留位置。空组计数为零。最终保留数组将未使用槽位填为 $(+\infty, -1)$，残差精排跳过负位置。这两种哨兵用途不同：负位置填充项不能作为有效 key 参与合并。当 $G = 1$ 时，直接写最终保留数组，无需合并。

较早的查询级路径由实现标签 $G = 0$ 选择，表示每条查询一个 block，而不是零个 block。其默认主码选择按分数精确，但仅比较距离的截断规则可能在并列处保留与完整 key 路径不同的成员。有损的 `K_LOCAL=4` 替代路径需要在允许该路径的构建中通过 `JHQ_EXACT_TOPCK=0` 开启；paper-faithful 构建强制使用精确选择。它不是被测默认路径。第7.4节 报告了 64 组分组扫描比较：主码选择没有非并列差异，在实际使用查询上最终输出一致。这些检查支持覆盖范围内的实现行为；本命题证明的是有条件的选择性质，不是原空间精确 kNN，也不是与所有旧并列规则无条件等价。

5.5 共享完整 Key 阈值与执行调度

保留完整局部 top-c 建立了正确性基础，但为了得到 c 个全局输出，可能生成 Gc 个中间候选。共享界组件 v63 减少这部分多余的选择工作，调度器 v65 再决定暴露多少扫描并行度。二者均在给定精排预算下工作；选择预算是第6章讨论的独立问题。

5.5.1 带明确见证的完整 Key 上界

当 $|U| \geq c$ 时，用 $K_{(c)}$ 表示全局第 c 小的 key。若某组持有 c 个互不相同且已完整评分的候选，则这些见证中的最大 key τ_g 满足 $K_{(c)} \preceq \tau_g$：全局集合中已经存在 c 个不晚于 τ_g 的候选。共享界初始化为最大哨兵，并按以下方式更新：

$$\tau \leftarrow \min_{\prec}(\tau, \tau_g). \tag{5.11}$$

不足 c 个有效候选时不得发布界。如果整个候选集合不足 c，任何组都无法发布这样的界，所有有效候选保持可接纳。

命题 5.2 (由见证界安全排除候选)**.** *假设 key 固定，每次发布都有 c 个互不相同的候选作为见证，每次读取返回初始哨兵或某个完整的已发布 key。仅在 $K_q(x) \succ \tau$ 时排除候选，不会删除任何全局 top-c 成员，与发布时序无关。*

证明. 每个已发布界都不小于 $K_{(c)}$，这些界的最小值也不小于 $K_{(c)}$。全局优胜候选满足 $K_q(x) \preceq K_{(c)} \preceq \tau$，因此不会被排除。较旧的界仍是上界，只会允许更多工作。此前的安全排除也不破坏后续发布的有效性：任意保留的 c 个互不相同候选仍提供 c 个见证，无论它们是否是该组最好的 c 项。 □

最后这一点很重要。启用共享后，一个组不必返回自己的完整局部 top-c：它可以丢掉局部有竞争力、但已被证明不可能进入全局结果的候选。正确性只要求各组输出的并集仍包含全部全局优胜候选，之后命题 5.1的截断合并论证仍适用。与共享界相等的候选继续有资格被接纳，包括建立该界的见证候选。实际接纳条件为

$$K_q(x) \prec \lambda_g \quad \text{and} \quad K_q(x) \preceq \tau_{\text{seen}}, \tag{5.12}$$

其中 λ_g 是局部压缩阈值，τ_{seen} 是该组最近一次观察到的共享界。

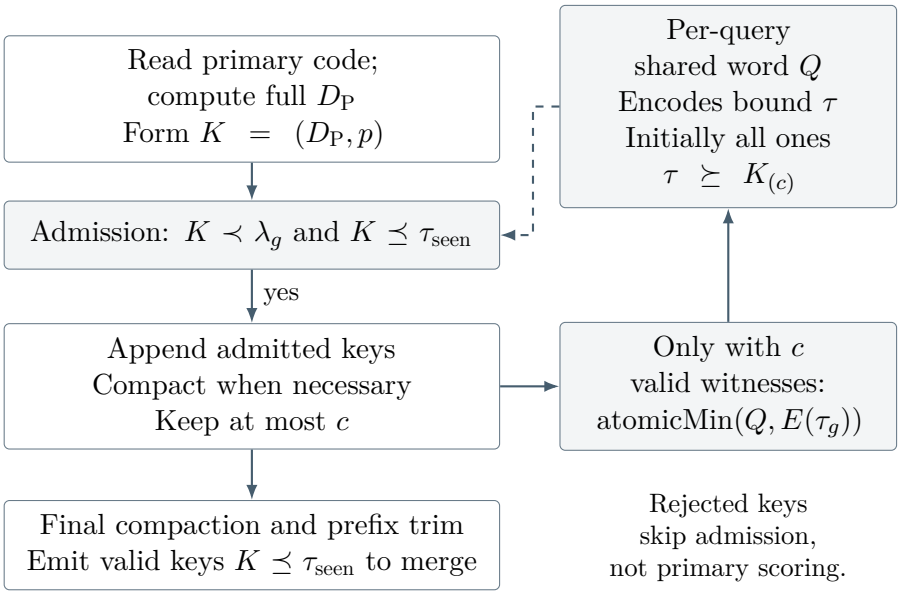


图 5.4: 单个扫描组的共享完整 key 过滤。候选的完整主码分数在接纳之前计算。有效的 c 候选见证集合可以收紧查询级共享界；陈旧但完整的界只会削弱过滤。最终前缀裁剪减少合并的有效输入，但不改变已分配工作区的容量。完整 key 读取这一前提的实现边界在正文中另行说明。

5.5.2 发布、过滤与实现边界

对于由平方和生成的非负 FP32 分数，代码将距离位模式和非负位置打包为无符号 64-bit key：

$$E(D, p) = 2^{32}\,\mathrm{uintbits}_{32}(D) + p. \tag{5.13}$$

此时无符号整数比较与字典序一致。这依赖正零编码，并排除 NaN 和负分数；它不是通用的有符号浮点编码。正无穷排在有限分数之后，全一位模式的初始字又排在有效编码之后。无效位置不得用于发布界。这些是分数定义域与表示的前提，不能从 recall 不变推断出来。

每个 block 的一个线程在每个 chunk 读取该查询的共享界。压缩后，如果有效条目足够，就通过 64-bit `atomicMin` 发布局部第 c 小 key。扫描结束时，再进行一次发布，并通过二分查找只保留不晚于观察到的共享界的有序前缀。各 block 不进行跨 block 屏障同步，也不等待其他组。较晚的更新可能使另一组多输出一些条目，精确合并会安全删除它们。每次搜索前在同一 stream 中重置共享界，扫描 kernel 完成后才执行合并。

读取前提需要单独限定。归档代码以原子操作发布界，却通过 `volatile` 64-bit load 轮询。CUDA 不保证 `volatile` 访问具有原子性或线程间同步语义 [7]。要可移植地实现上述命题，读取端也应使用设备作用域的原子读取。第7.4节中记

录的 48 组配对检查支持被测构建中的实测一致性，不构成语言内存模型层面的并发证明。原子读取修订及其 GPU 复验尚未执行。

图 5.4也明确了优化省下什么。每个被扫描候选仍读取主码，并计算完整主码分数。共享界避免一部分缓冲区追加、局部排序和中间写出，并缩短合并输入列表。它没有实现基于部分距离的提前终止，也没有减少完整主码评分的候选数量。收益还要抵消轮询和原子更新的开销；第7.4节报告的配对加速范围中也包含轻微变慢的情况。

5.5.3 在不改变预算的情况下选择并行度

对 batch 大小 b，分组扫描启动 bG 个 block，而不是 b 个。这可以暴露原本不足的并行度，但也重复 LUT 搬运和局部选择，并增加合并步骤，因此固定组数的设计不可能在所有情形下都更好。较小静态配额可以减少中间存储，但要保持覆盖，需要额外证书或回退；pilot 观察到的优胜候选集中现象促使系统选择带见证的阈值方案（第7.4节）。

实际调度器结合并行填充的近似指标和工作负载偏斜统计。令 T 为扫描 block 的线程数，H 为 SM 数乘以每个 SM 的最大驻留线程数，$v = \mathrm{sd}(n_c)/\,\mathrm{mean}(n_c)$ 为候选数的变异系数。规则为

$$G_{\mathrm{fill}} = \max(\{1\} \cup \{g \in \{2,4,8\} : bgT \le 0.6H\}), \tag{5.14}$$

$$G_{\mathrm{skew}} = \begin{cases} 4, & v \ge 0.3, \\ 1, & v < 0.3, \end{cases} \qquad G_{\mathrm{chosen}} = \max(G_{\mathrm{fill}}, G_{\mathrm{skew}}). \tag{5.15}$$

偏斜规则可能覆盖填充限制。实验程序以 512 个数据库向量作为伪查询，在固定路由配置下校准 v，不使用 ground-truth 标签。这与第 6 章的采样输出一致性校准不同。校准之后，调度只依赖 batch 大小和已保存统计，不增加每次搜索的主机读回。两个常数在四个 pilot 配置上拟合，之后再到其他配置上评估。

当 $G_{\mathrm{chosen}} = 1$ 时，实现选择更快的查询级路径，标签为 $G = 0$。它不改变索引、探针数或精排预算，但继承该路径仅按距离截断时的并列行为。因此，在声称所有调度选择产生相同保留集合之前，必须检查共同全序前提。第7.6节报告独立的集成输出检查，以及调度器的实测收益和 regret。

该规则是启发式策略，不是 occupancy 计算，也不是延迟 oracle：它没有建模寄存器压力、共享内存驻留限制、与预算相关的压缩成本和合并成本。冻结的伪查询统计也可能无法代表未来查询。记录中的最大 regret 出现在 batch 256，此时某个固定组数方案约快 1.217×。因此，这里的设计贡献是经测试的候选并行扫

描、带见证过滤与工作负载相关执行的组合，不是新的通用 top-c 恒等式，也不保证最优调度。后续残差精排与最终 top-k 按第5.6节消费这些有效候选。

5.6 Lane 协作的残差评分与最终 Top-k

只有主码保留候选才访问残差码。这使系统先对 n_c 个候选进行广泛而便宜的扫描，再执行较窄的 $O(cd)$ 评分阶段。如果对每个访问候选读取残差，坐标计算量将趋向 $O(n_c d)$。lane 协作加速已选候选，不负责决定精排预算。

5.6.1 重建评分与查询相关常数

令 $\widehat{p}(x)$ 为主码重建，$\widehat{r}(x)$ 为变换坐标中的残差解码。编码器存储 $\gamma(x) = 2\langle \widehat{p}(x), \widehat{r}(x) \rangle$。实现中的精排排序分数为

$$D_{\mathrm{H}}(q,x) = D_{\mathrm{P}}(q,x) + \|q' - \widehat{r}(x)\|_2^2 + \gamma(x) \tag{5.16}$$

$$= \|q' - \widehat{p}(x) - \widehat{r}(x)\|_2^2 + \|q'\|_2^2. \tag{5.17}$$

第二个等式由展开平方范数得到。因此，该分数按平方距离对层次重建排序，但返回值还包含一个仅与查询有关的常数；它既不是直接计算的原始向量距离，也不是未平移的重建距离。该常数不影响固定查询的排序。即使按此分数精确选择，重建误差仍然存在 [4]。

残差坐标使用子空间内共享的标量码本项。标量路径遍历每个候选的各维，累加与 q' 的平方差。查询坐标及在容量允许时的残差码本会搬入共享内存，最后再加上已存的修正项和主码分数。

5.6.2 Lane 到 word 的映射与部分和归约

对 $\ell_{\mathrm{ref}} = L \in \{1, 8, 16, 32\}$，独立的精排 block 为每个候选分配 L 个相邻 lane。一个 32-bit 残差 word 包含 $R = 32/B_R$ 个坐标编码，其中 B_R 为残差位宽。被测八比特路径中 $R = 4$，四比特路径中 $R = 8$。lane a 读取 word 索引 $a, a+L, a+2L, \ldots$，解码坐标并累加部分和。butterfly shuffle 归约合并 L 个部分和，lane zero 写出精排分数。

启动条件要求残差行按 word 对齐，且 D_s 能被 R 整除，从而一个 word 不跨越两个子空间的码本边界。所有 lane 都执行 shuffle 序列，包括分配到无效或超出范围槽位的 lane；它们在输出标为无穷前贡献零。这避免让 warp 参与情况取决于某个候选是否存在。$L = 16$ 时，一个 warp 处理两个候选，而不是每个 lane 各处理一个候选。

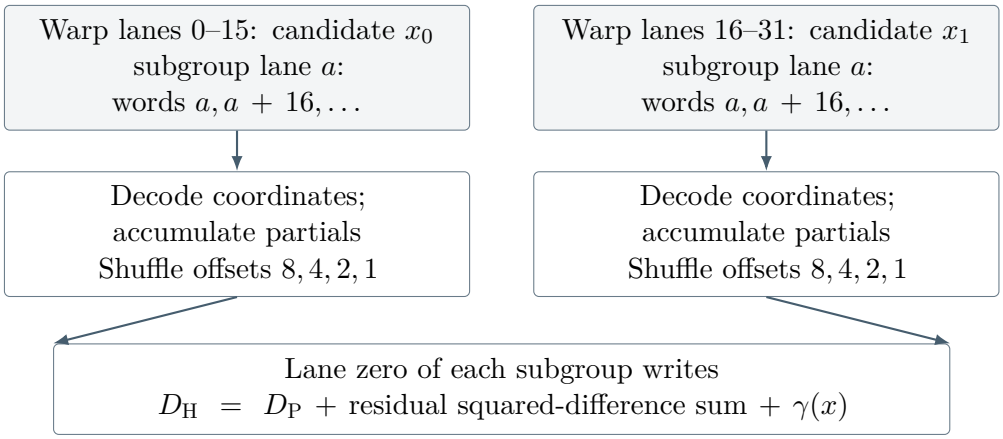


图 5.5: 十六 lane 残差精排。一个 warp 处理两个候选；候选内的 lane a 读取残差 word $a, a+16, \ldots$。残差编码为八比特时，每个 32-bit word 解码四个坐标。子组归约合并部分和，再加上保留的主码分数和修正项。即使子组内 word 访问连续，候选物理位置仍可能分散。

实验将两种改变分开。L0 是原来的标量逐字节加载路径，L1 使用 word load，但仍每候选一个 lane，保持其累加顺序。L8/L16/L32 进一步将坐标工作分配给多个 lane，缩短每个 lane 的串行遍历，并在 c 较小时提供有用工作；代价是增加 shuffle，并减少同时处理的独立候选数量。因此 L 并非越大越好。保留候选位置不规则、码本处于共享还是全局内存，也仍影响成本。

5.6.3 最终选择与数值边界

当至少有 k 个有效候选具有有限精排分数、且数据库 ID 唯一时，最终 kernel 按 $(D_{\mathrm{H}}, \text{database ID})$ 精确选择 top-k。每条查询一个 block，读入 c 个分数后反复归约最小值，输出其 ID 和分数，再移除该分数进入下一轮。每轮扫描全部 c 个槽位，没有固定的逐线程候选配额。这一正确性陈述不包含少于 k 个有限有效候选的退化池最终输出约定。

最终 ID 并列规则与主码保留使用的物理位置规则不同。精确的最终选择器不能找回路由或主码筛选丢掉的候选，也不能消除量化误差。多 lane 归约在选择前改变浮点结合顺序，因此即使实数公式不变，近似并列候选也可能发生排序变化。

第7.3节区分这些影响：L1 在被测用例中逐位一致，多 lane 的距离相对误差最高约为 1.6×10^{-6}。普通预算下 L16 端到端收益范围为 1.015–1.789×；内核级收益更大，不能替代端到端数字。受控的 load-only 与协作变体支持设计结论，而数值等价性需要检查，不能预先假定。

5.7 作为独立原语的 IVF 探针选择

探针选择发生在主码扫描之前，选择的是倒排列表，而不是数据库向量保留候选。对 IVF 中心 μ_i，选择器对以下计算所得 FP32 路由分数排序：

$$s_i = \|\mu_i\|_2^2 - 2\langle q', \mu_i \rangle, \qquad (s_i, i) \text{ ordered lexicographically.} \tag{5.18}$$

省略共同的查询范数不改变中心排序。与主码平方距离表不同，这些分数可以为负。所需输出是按分数排序的前 $P = \min(n_{\text{probe}}, n_{\text{list}})$ 个中心 ID，并列时较小列表 ID 优先，以及它们列表长度的前缀偏移。

5.7.1 Radix 选择与确定性排序

v65 选择器对逐渐填充的候选缓冲区反复执行 bitonic 排序。v69 替代方案仍为每条查询使用一个 block，但通过四轮从最高有效字节开始的 radix pass 确定截断值。每轮只统计匹配此前已选前缀的 key，再找到包含剩余顺序统计量的 bin。四轮之后，得到 cutoff key 及仍需选择的同 cutoff 条目数。

浮点映射先将正负零均规范化为正零，再对 32-bit 表示 u 作变换：

$$F(u) = \begin{cases} \sim u, & \text{if the sign bit is set,} \\ u \mid 2^{31}, & \text{otherwise.} \end{cases} \tag{5.19}$$

对非 NaN 浮点数，它保持数值顺序，包括负分数和无穷；规范化使相等的零具有同一 key。这与式 (5.13) 的非负分数编码不同。NaN 不属于选择器的排序契约。每轮使用相同、明确舍入的 `fma` 表达式重新计算分数，因此轮次改变不会改变 key。

直方图有 256 个 bin 和 32 份 lane 副本，仅计数器就占 32 KiB。副本将同一 warp 内对热门 bin 的原子更新分开，不同 warp 之间仍可能竞争。这是以共享存储和归约工作换取更分散的直方图更新，不是消除全部竞争。

收集阶段先取得所有小于 cutoff 的 key。如果等于 cutoff 的 key 过多，就按 ID 顺序重新扫描，通过 ballot 和前缀计数选取恰好所需数量、且列表 ID 最小的成员。因此最终并列子集不由原子到达顺序决定。随后，对 P 个已选条目分别统计有多少其他已选 key 在 (s_i, i) 顺序中排在它之前，再写入相应名次。这无需排序全部中心，就恢复了确定性的有序输出。

最后，block 前缀扫描生成所选列表长度的偏移和总候选数。当 $n_{\text{probe}} > n_{\text{list}}$ 时，实现通过重复最后一个已选 ID 填充 ID 数组，但将多余条目的长度置为零，不会再次扫描同一列表。这里假设中心集合非空，且请求探针数为正。

表 5.2: 各选择阶段的对象、并列规则和 reference checks 不同。一个阶段的证据不能建立另一阶段的正确性。

阶段	排序对象	并列 key	对应证据
探针	中心路由分数	列表 ID	合成 ID、偏移与总数；真实输入 dump
主码	已访问候选的主码分数	分组路径中的物理位置	local / merge 证明；独立界 / 共享界检查
最终	保留候选的精排分数	数据库 ID	全 c 选择代码；残差 / 输出比较

5.7.2 成本与独立正确性契约

直方图和收集各轮会重复访问中心分数行；名次统计需要 $O(P^2)$ 次比较。因此该算法不是对任意探针数都具有统一线性成本的替代品。较大的 P 同时增加排序工作与已选条目的共享存储，而中心集合较小时也仍需预留直方图空间。Radix selection 是已有选择技术；这里的贡献是它在该查询流水线中的确定性集成及实测执行权衡。

归档探针 checker 报告 1,792 个用例，与早期选择器比较 ID、偏移和总候选数，差异为零。输入分布覆盖大量并列、全部相等、混合正负零、大部分为无穷和极大量级；还覆盖请求探针数大于列表数的情况。这是针对探针原语的有限测试，不是主码 top-c 或原空间精确近邻的证明。

第7.4节给出真实查询一致性检查、阶段计时和端到端对照，也保留了 Vogue 跨进程异常：probe / primary dump 匹配不意味着最终流水线输出必定相同。可选 v70 FP16 路由改变分数精度，属于独立近似；探针高重合度或 recall 不变，都不能使其归入 FP32 等价性主张。

第 6 章

Workload-Level Refinement-Budget Calibration

待撰写： Define the policy, its reference outputs and assumptions, then separate sampled agreement, held-out quality, and calibration economics.

6.1 Why Fixed Refinement Budgets Can Be Wasteful

待撰写： Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

6.2 GT-Free Sampled Output Agreement

待撰写： Define the disagreement-slot sum and distinguish it from a per-query rate. Explain how fixed epsilon changes strictness as the sample size changes.

6.3 Budget Search, Nestedness and Tie Safety

待撰写： Check nested survivor sets, fixed refined scores, exact-selection configuration, deterministic ties, and the executed path before invoking monotonicity.

6.4 Budget Baselines and Their Questions

待撰写： Explain the distinct questions answered by fixed budgets, same-sample enumeration, full-pool calibration, and the ground-truth oracle.

6.5 Hold-out Risk, Calibration Cost and Payback

待撰写： Separate selected-arm throughput from total cost. State held-out quality loss, sampling variability, calibration overhead, and reuse assumptions.

6.6 Policy Failure Modes and Applicability

待撰写： Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

第 7 章

实验评估与研究发现

本章通过构建测量、受控执行改动、留出查询预算选择和跨系统搜索，回答论文的四个研究问题。对于 **RQ1**（构建与容量），最大被测索引在单卡上搜索 17.78 百万个向量，常驻设备显存为 20.44 GiB；对应的冷训练加添加记录为 10.40 s。对于 **RQ2**（正确且高效的 GPU 执行），两组查找表在 22/24 个配置中领先；批大小 8 时，调度扫描的吞吐量几何平均提高至 2.727×，并保持被测最终标识符集合一致。对于 **RQ3**（预算质量与净成本），所选配置的平均 QPS 在 10/12 个单元中优于固定 $\alpha = 100$，在 9/12 个单元中优于固定 $\alpha = 64$，但最多 40/64 次校准抽样超过相对 oracle 的召回率损失阈值。对于 **RQ4**（竞争区间），召回率 0.97 处，插值比较中 JHQ 在三个 E1 数据集上最快，在 arXiv 上与 IVF-RaBitQ 数值接近；CAGRA int8 则在较低召回率领先。

7.1 最终实验清单与对照实验组

实验分组区分了回答 RQ1–RQ4 所需的证据。组件名称描述机制，版本号标识实现来源。记录中的运行源码版本、二进制名称以及源码到结果的映射见第A章。

7.1.1 证据分组与测量边界

表 7.1 列出了本章使用的实验组。版本号表示不同的实验实现，而不是对同一个不变基准的连续测量。尤其需要区分历史 ADC 论文与后来的 v69/v71 实验：它们的计时结果不同。本章相应的数字以较新的原始记录为准；历史打包访存和精排实验则作为独立消融保留。将这些实验的加速比直接相乘，会混合不同基线，得不到实际测量过的系统加速比。

E1 的环境记录为 NVIDIA GeForce RTX 5090，设备报告显存 32,607 MiB，驱动 595.71.05，CUDA 13.0，cuVS 26.08.01。容器的 CPU 配额为 25 核，OpenMP 限制为 16 个线程。JHQ 使用 Release 构建，先编译为 compute_80 PTX，再由驱动针对设备即时编译；基线使用已安装的原生 cuVS 库。

跨系统比较的主要计时边界为“主机输入到主机输出”：从主机内存中的 FP32 查询开始，到 top-10 标识符和距离返回主机内存、流同步完成为止。它包含数据传输、查询预处理与搜索；CAGRA int8 的查询量化也包含在内。基线复用资源

表 7.1: 实验与证据分组。数量表示测量配置或检查次数，不代表独立索引构建次数。所有实验均使用 RTX 5090，但计时协议有所不同。

实验组	目的与实现	覆盖范围
E1	v69 与 cuVS 图索引、PQ、RaBitQ、暴力搜索比较	1,590 个点；4 个数据集
LUT	v71 查找表分组，$g_{\mathrm{LUT}} \in \{1, 2, 4, 8\}$	24 个单元；96 个变体
打包访存	历史字节/机器字访存析因对照	48 个配对单元
精排	v61 标量、加载方式与 lane 数消融	13 个配置
扫描/共享界	v62 全局选择；v63 共享界	64 / 48 个正确性比较
调度	集成后的 v65 执行策略	50 个工作负载—批大小单元
探针选择	v69 精确 FP32 探针选择	1,792 个合成用例
预算	v69 抽样、全校准池与 oracle 预算	12 个单元；每个 S 抽样 64 次
规模	v69 搜索、冷构建与常驻显存	BGE 与 Stella 扩展实验

句柄和输出缓冲区。E1 使用五轮计时，每轮通常调用 24 次。当包含首次调用的预热区间超过每次调用的阈值时，基线程序减少调用次数；274 个点采用了较少调用，其中也包括稳态速度较快的点。报告的 QPS 是各轮吞吐量的中位数，每轮吞吐量由总查询数除以总耗时得到，而不是单次调用延迟中位数的倒数。预热、索引构建以及任何预算校准均不计入。基线的纯设备端时间仅作为辅助结果：JHQ 没有等价的设备端查询接口，将这些时间混入主要比值会改变比较口径。

7.1.2 指标与可复现性

所有实验均采用平方 L2 距离和 $k = 10$。召回率是找回真实 top-10 标识符的平均比例；即使 ground-truth 文件每行存储了更多结果，也只比较前十项。精排系数决定候选保留数 $c = \max\{k, \lceil \alpha k \rceil\}$。本章中，$g_{\mathrm{LUT}}$ 表示笛卡尔查找表分组数，g_{scan} 表示并行扫描组数；这两个参数控制不同的工作。

本章分析脚本只读取已有测量，不运行新的 GPU 实验。它核对记录数量与正确性字段，重新计算配对比值和匹配召回率结果，并直接从原始记录生成数字表格。配套证据清单记录了所用文件的 SHA-256 哈希。其中的源码检出版本号表示本次核对所使用的仓库快照，并不意味着所有历史二进制程序都由这个提交构建。各次运行的原始日志头和实验报告仍保留其各自构建来源。这样既能复现当前图表，也不会错误地给历史实验重新标注版本。[1]

7.2 现代 GPU ANN 的召回率—QPS 前沿

本节通过定位共同召回率下的运行区间回答 RQ4，同时包含竞争方法更快的区间。

[1]报告仓库提供 `scripts/build_ch7.py`、`data/ch7/audit.json`、`data/ch7/matched.csv` 和证据清单 `data/ch7/sources.csv`。脚本需要同级目录中包含归档测量的 `JHQ_GPU` 仓库。

7.2.1 比较范围与前沿构造

图 7.1 将 JHQ-GPU 与 CAGRA fp32、经过标量量化的 CAGRA int8、IVF-RaBitQ、IVF-PQ 和 GPU 暴力搜索进行比较。CAGRA 代表图搜索；PQ 和 RaBitQ 代表使用压缩表示的倒排文件方法 [5, 8, 9]。四个数据集分别为 Vogue（$932{,}328 \times 768$）、arXiv（$2{,}253{,}000 \times 768$），以及各含 999,000 个向量、维度分别为 1,536 和 3,072 的两个 OpenAI 数据集。每个数据集使用 1,000 条查询，各方法共享同一份 ground-truth 文件。

每种方法贡献其非支配的实测运行点。如果同一方法的另一配置在召回率和吞吐量上均不差、且至少一项严格更好，则该点被剔除。因此，一条前沿组合了不同搜索参数，有时还组合了不同索引配置；它并不一定表示同一个固定索引的变化轨迹。每个数据集上，JHQ 提供 27–39 个测量点，各基线组提供 54–264 个点。E1 解析器得到 1,590 个有效点，没有失败记录，也没有无效标识符或行内重复标识符标记。这个数量表示参数扫描的规模，而不是 1,590 次独立重复实验。

参数覆盖较为充分。CAGRA 改变图出度 32/64、搜索宽度 1–8 和中间 top-k 容量 32–512。RaBitQ 改变 4/8/9-bit 预算、支持的搜索模式以及最高 1,024 的探针数。PQ 改变编码长度、查找精度和探针数。JHQ 同时改变探针数与精排预算，并在初始网格无法充分覆盖召回率时扩展网格。比较结论因而受限于这些已记录的参数网格。某条曲线在目标召回率之前结束，表示该区间没有测量，不能证明该方法无法达到目标。

这些曲线比较的是可达到的吞吐量，而不是相同显存占用下的表现。JHQ 每个向量约存储 $d/8$ 字节主码、d 字节残差码以及元数据。九比特 RaBitQ 的名义编码预算为 $9d/8$ 字节，与 JHQ 接近，但它的 4/8-bit 配置使用更少存储。CAGRA int8 还存储图边；PQ 编码长度可变；CAGRA fp32 保留 FP32 向量。对这些选项取性能包络有助于部署选择，但不能单独证明相同显存预算下的表示优势。显存证据在第7.7节另行讨论。

7.2.2 匹配召回率与不同运行区间的反向结果

表 7.2 使用两种估计方式，量化召回率 0.97 附近的表现。插值估计在相邻前沿点之间对吞吐量取对数后进行线性插值，不向覆盖范围之外外推。实测阈值估计选取达到目标召回率的最快采样点。插值估计将与目标相差不超过 5×10^{-4} 的实测召回率视为匹配；实测阈值估计则要求召回率至少达到目标。插值减小了参数网格间距不一致的影响；实测阈值估计对应真实记录过的配置。两者都不能凭空产生一个召回率恰好等于目标的新测量。

相较于 CAGRA fp32，该目标下的插值比值从 arXiv 的 1.317× 到两个 OpenAI 数据集的 1.747× 不等。相较于 IVF-RaBitQ，对应数值分别为 1.059×、

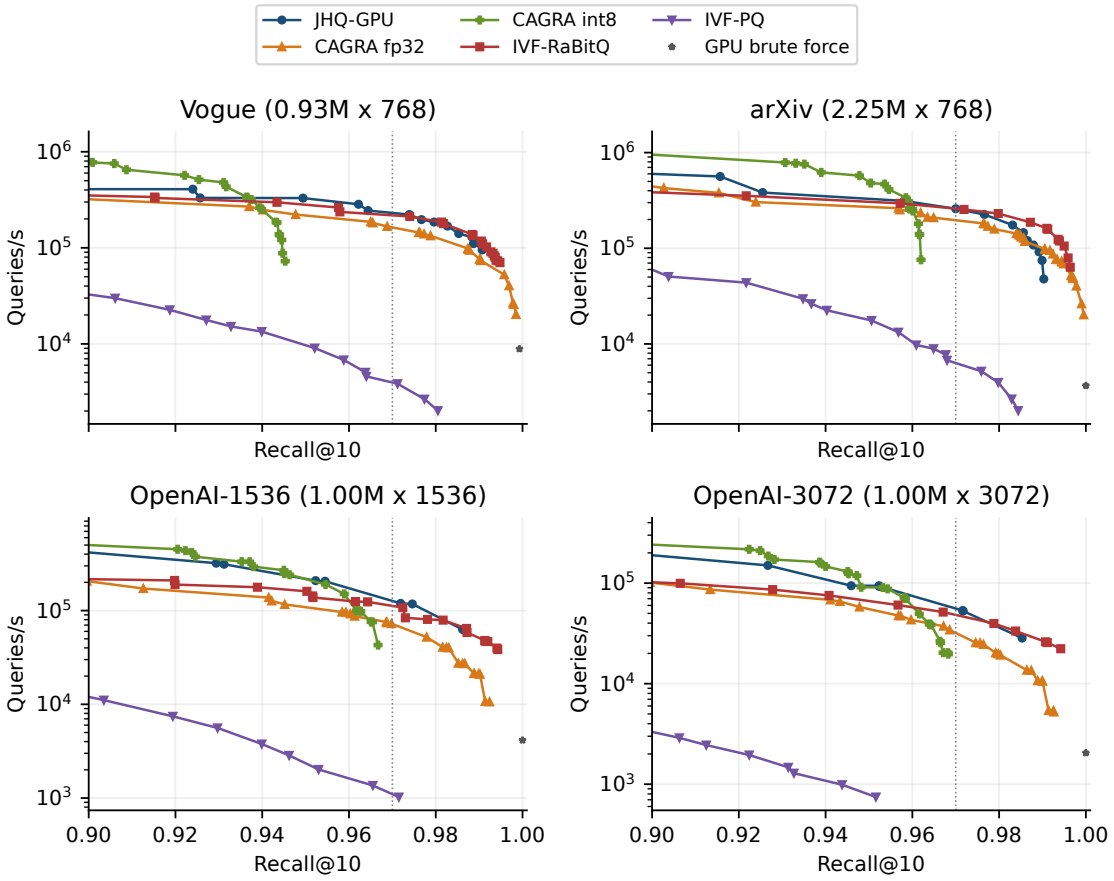


图 7.1: 四个 E1 数据集上实测的主机到主机召回率—QPS 前沿。图中从召回率 0.90 开始显示，并在 0.97 处标出竖线；构造前沿时纳入了包括 CAGRA int8 在内的全部基线组。连线连接实测点，吞吐量轴为对数坐标。

表 7.2: 召回率 0.97 下的比较。每组比值依次为 “插值 / 实测阈值”；大于一表示 JHQ 更快。JHQ kQPS 为插值估计。arXiv 与 RaBitQ 的微小差异随估计方法而改变方向。

数据集	JHQ kQPS	相对 CAGRA fp32	相对 IVF-RaBitQ
Vogue	230.8	1.416 / 1.537	1.059 / 1.046
arXiv	258.6	1.317 / 1.422	0.996 / 1.015
OpenAI-1536	126.7	1.747 / 2.294	1.130 / 1.111
OpenAI-3072	55.9	1.747 / 2.088	1.157 / 1.346

0.996×、1.130× 和 1.157×。arXiv 的结果应视为接近平手：插值吞吐量略低，但实测阈值比较略高。如果将其表述为明确获胜，就会让插值约定获得超出测量本身所支持的证据权重。

在有共同覆盖的比较中，目标召回率 0.90–0.985 范围内，JHQ 相对 CAGRA fp32 的插值优势约为 1.10–2.44×。但是，加入 CAGRA int8 后，低召回率区间的解释发生变化：在 0.90–0.92 附近，其低精度图搜索可以快于 JHQ。在更高召回率下，IVF-RaBitQ 常重新领先，包括若干 0.98 及以上目标。在 arXiv 约 0.99 的召回率处，记录网格中的 CAGRA fp32 也快于 JHQ。因此，可被证据支持的结论是：JHQ 在部分有价值的区间具有竞争力，尤其是中高召回率附近，而不是全面优于现代 GPU ANN 方法。

CAGRA int8 在 Vogue、arXiv、OpenAI-1536 和 OpenAI-3072 上的最高实测召回率分别为 0.9453、0.9620、0.9667 和 0.9683，因此这些网格中没有达到 0.97 的配置。按相同插值规则，目标召回率 0.90–0.92 处，它约比 JHQ 快 1.20–1.98×；在 arXiv 的 0.95 和两个 OpenAI 数据集的 0.94 处也仍然更快。0.97 的结果针对具有测量覆盖的方法，不能推广为低于 0.97 的每个目标均领先。

网格稀疏时，两种估计可能有明显差异。例如，OpenAI-1536 在召回率 0.97 处相对 CAGRA fp32 的比值，插值为 1.747×，而按满足阈值的实测点计算为 2.294×。后一个数值包含了基线下一个超过阈值的测量点所付出的代价，不能将其表述为在完全相同召回率下直接测得的同等幅度优势。配套匹配表保留两种估计，并标明没有覆盖的目标，使读者能够区分真实的运行区间结果与参数采样带来的影响。

7.2.3 结果解释与比较限制

JHQ 将低成本主码筛选与选择性残差访问结合。当只需精排少量已扫描候选时，这为其有竞争力的吞吐量提供了合理解释。后面的内核与预算消融直接检验这一解释的部分环节。仅靠跨系统曲线，不能将收益归因于某个单一组件，因为各方法在路由、表示、预处理、索引构建和软件接口上均有差异。尤其是，数据集维度提高的同时，数据分布与规模也在变化。因此，OpenAI 数据集上更大的比值，并不是“维度提高导致 JHQ 优势扩大”的受控证据。

即便对齐主要计时边界，比较中仍存在限制。JHQ 从 C++ 调用，CAGRA 和 PQ 则通过 Python 绑定调用；即使复用资源，主机端开销仍可能有小幅差异。JHQ 输出回传至可分页主机内存，基线使用页锁定内存。编译目标不同，无关的主机负载也无法完全控制。对于很小的性能差距，这些因素不可忽略。五轮测量描述的是已构建索引上的重复执行，不能估计独立重建索引或换一台机器后的变化。

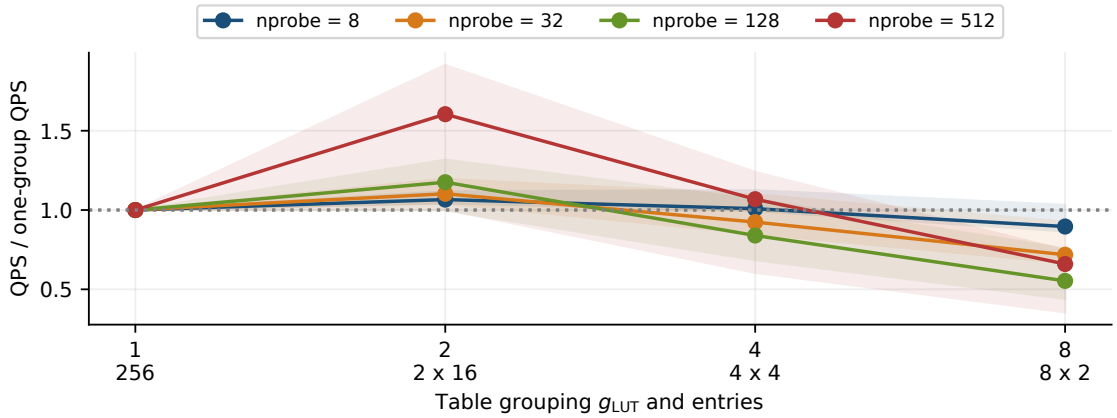


图 7.2: 固定 $\alpha = 100$ 的笛卡尔查找表分组实验（v71）。组数表示 g_{LUT}，不是并行扫描组数。曲线为各数据集相对单组 QPS 比值的中位数，阴影表示范围。

7.3 笛卡尔查找表、打包加载与 lane 协作精排

针对 RQ2，消融区分了保持表示不变的查找表、加载改动与协作式残差评分，并同时测量内核效果和整体应用效果。

7.3.1 查找表的精确因子化及其吞吐量影响

笛卡尔因子化改变的是已表示主码距离的计算方式，而不是重新训练一个量化器。将一个八比特笛卡尔编码拆为两个四比特分量，可把 256 项查找表替换为 $2 \times 16 = 32$ 项。对于 FP32 表项，每个对应查找表由 1,024 字节降为 128 字节，即缩小八倍。数学上的评分保持不变，但浮点求值顺序仍可能改变末尾比特。因此，仅凭查找表更小，不能断言完整 ANN 输出逐位一致。

图 7.2 比较六个数据集上的笛卡尔查找表分组，探针数为 8/32/128/512、固定 $\alpha = 100$，以及 $g_{\mathrm{LUT}} \in \{1, 2, 4, 8\}$。每个变体有五轮计时；除 Stella 使用 999 条查询外，批大小均为 1,000。在 24 个单元中，两组查找表有 22 个取得最高吞吐量中位数。例外是 arXiv 的 128 探针配置，单组约快 0.23%；以及 OpenAI-3072 的八探针配置，四组约快 0.84%。这些小差异更适合视为接近平手。新版统计是 22/24，而不是历史 ADC 论文中的 23/24。

两组相对单组的吞吐量比值为 0.998–1.914。四组与八组都只需总计 16 个表项，但查表次数和粒度不同。这构成了有用的对照：在两组设计的基础上进一步缩小表存储，并不必然提高搜索性能。额外的索引计算与累加工作可能超过存储节省带来的收益。测量建立的是净收益与开销之间的权衡；缺少独立的占用率或指令计数实验，就不能将某个硬件因素认定为唯一原因。任一四变体单元内，报告的最大召回率差为 10^{-4}。

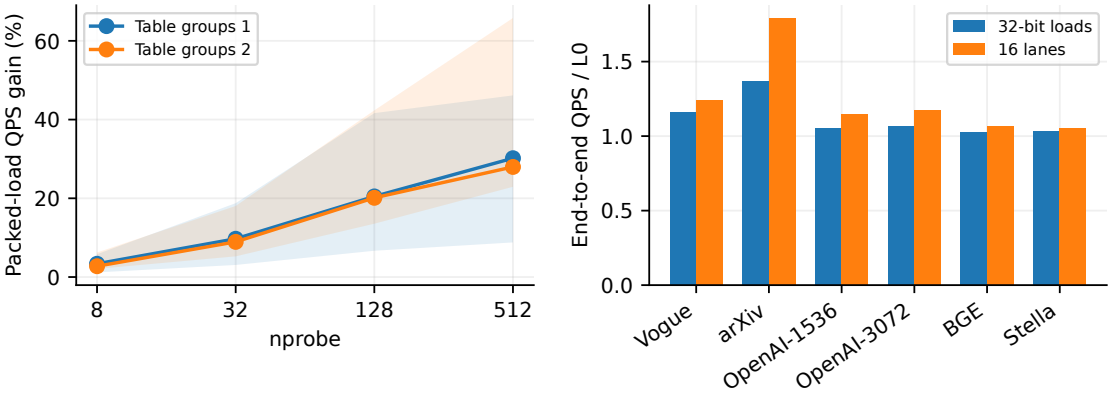


图 7.3: 打包访存与精排对照。左：各查找表分组和探针数下，六个数据集打包加载 QPS 增益的中位数，阴影为最小值至最大值。右：128 探针及各数据集普通预算下，仅改变加载和使用 16 个 lane 的实测端到端收益。两个子图来自不同实验组。

表 7.3: 128 探针下的精排结果。除最后一列为剖析得到的精排内核加速比外，其余比值均为相对 L0 的配对端到端收益。不能将内核加速比与应用整体加速比混用。

数据集	α	L1/L0 吞吐比	L16/L0 吞吐比	内核加速比
arXiv	100	1.368	1.789	5.97
BGE	32	1.026	1.062	1.97
OpenAI-1536	8	1.055	1.145	3.34
OpenAI-3072	4	1.063	1.171	8.81
Stella	16	1.030	1.053	2.66
Vogue	64	1.161	1.240	3.72

7.3.2 打包访存与协作式残差评分

历史打包实验固定数据集、探针数和查找表分组，比较以字节和机器字为单位的访存。在六个数据集、四个探针数以及 $g_{\mathrm{LUT}} \in \{1, 2\}$ 的 48 对比较中，吞吐量比值为 1.012–1.658，最大召回率差为 10^{-4}。八探针时收益通常较小，512 探针时较大。两种实现都已经采用子空间优先布局，因此，这个实验支持打包访存的效果，而不是“仅靠物理转置就取得了提升”的说法。每个变体只报告一个汇总吞吐率，因而这一较早证据提供的不确定性信息弱于五轮计时的 v71 扫描。

图 7.3与表 7.3 进一步区分了残差精排中更宽加载与 lane 协作的作用。L0 是原始标量路径；L1 改变加载方式，但不将单个候选分给多个 lane；L16 则由 16 个 lane 协作处理一个候选。同一进程在五轮计时中轮换变体顺序，每轮每个变体执行三次搜索。在十二个普通配置上，L16 的端到端吞吐量为 L0 的 1.015–1.789×。另外加入的 arXiv $\alpha = 200$ 配置明确属于高预算对照，达到 2.383×；不能把它并入普通所选预算的收益范围。

在被测用例中，仅改变加载的对照与 L0 逐位一致，但已使 arXiv 吞吐量提高至 1.368×。lane 协作进一步缩短了精排时间。在 arXiv 的 $\alpha = 100$ 配置上，精排内核由 4,277.5 降至 716.8 μs，即 5.97× 的内核加速，但整体吞吐量只提高至 1.789×。OpenAI-3072 的精排内核加速为 8.81×，整体吞吐量则为 1.171×。这与 Amdahl 定律一致：加速一个较小阶段，并不能消除路由、筛选、最终选择或传输的成本。这也说明了随后扫描与探针实验的动机——继续处理剩余工作。

定量 Amdahl 检验使用基线精排内核时间占主机到主机基线延迟的比例 f，以及内核加速比 s；若其余耗时不变，整体估计为 $1/(1 - f + f/s)$。arXiv 和 OpenAI-3072 的比例约为 0.529 和 0.168，对应估计 1.787× 和 1.175×，接近实测 1.789× 和 1.171×。它结合了单独的内核剖析与墙钟测量，只是机制解释的估计。

协作归约改变了浮点加法顺序。被测多 lane 变体的距离差异很小，最大报告相对误差约为 1.6×10^{-6}，但排序边界附近偶有结果变化。因此，证据支持的是数值接近、且具有实测端到端收益的评分，而不是无条件的逐位等价。Sanitizer 检查在较小查询子集上覆盖了记录中的精排和 top-k 内核；这些是有用的实现检查，并不是对任意输入的证明。

7.4 全局 top-c、共享界与探针选择的正确性及消融

本节先回答 RQ2 的正确性部分，再考察分组扫描、阈值共享和基数探针选择的成本。

7.4.1 先验证全局主码选择，再讨论性能

并行扫描实验处理了一个仅靠召回率无法评估的正确性要求：将同一查询的候选分配给多个线程块后，必须保持全局主码 top-c。如果每块只保留 c/g_{scan} 个候选，候选质量集中的分组可能丢弃本应进入全局结果的候选。JHQ 的分组扫描路径改为每组最多保留 c 个候选，再合并局部列表。选择使用完整排序键 $(D_{\text{P}}, \text{position})$，以位置确定性地打破主码距离并列。

局部保留规则有直接的正确性解释。一个属于全局 top-c 的候选，在整个候选集合中排在它之前的键少于 c 个，那么在自身所在分组中也一定少于 c 个。因此，它必然出现在该组的局部 top-c 中。对这些局部列表的并集使用相同全序选取 top-c，即可恢复全局 top-c。这一结论仅针对已访问候选上的主码评分；它不会使 IVF 路由变成精确搜索，不会将主码评分变成原始向量距离，也不会使最终 ANN 答案成为精确最近邻结果。

分组扫描检查覆盖四个工作负载、批大小 8/32/256/1,000，以及分组数 1/2/4/8，共计 64 个比较。记录显示，相对原始单线程块路径，没有非并列情况

下的主码选择差异，所比较主码距离的变化为零，而且每条实际使用查询的最终 top-10 标识符与距离逐位一致。原始路径只按距离排序，因此在主码截断边界发生并列时，可能选中不同成员；这并不违反新完整键的规范。在这一特定实验中，这些差异也未改变最终输出。

实际使用的查询数需要单独说明。测试程序只提交完整批次：从 1,000 条查询池中，批大小 32 使用 992 条，批大小 256 使用 768 条，而批大小 8 和 1,000 均使用全部 1,000 条。因此，正确性计数与计时样本随批大小而变化，不能把每一行都描述为相同的 1,000 查询测试。记录中的 memcheck、racecheck 和 synccheck 还对更小子集及指定扫描、合并内核提供了额外检查。

7.4.2 共享界保持选择目标不变

保留完整局部 top-c 是安全的，但成本可能很高。在共享界组件（v63）中，同一查询的各组通过相同完整排序键发布共享界。一旦某组已确定有 c 个键不大于该界，比该界更差的候选就不可能进入全局 top-c。过时、较宽松的界可能接纳更多候选，但不会因此错误地排除更优候选。这要求发布的界有足够数量的有效候选作为依据，并且原子更新保持完整键的顺序；仅有近似距离阈值，无法建立相同的并列处理语义。

实现仍会先为每个扫描候选计算完整主码评分，再决定是否接纳。共享界减少的是选择、压缩整理、排序与后续合并工作；它不会剪去主码距离计算，也不会避免读取所有被拒绝候选的主码。解释扫描阶段加速时，必须明确这一范围：扫描阶段变快，并不一定意味着计算了更少距离。

共享界检查，在四个工作负载与四种批大小上，将共享界的 2/4/8 组路径与相应独立界路径比较。48 个比较全部报告主码 top-c 和最终输出一致。五轮配对吞吐量相对独立界约为 0.994–1.562×。因此，共享界能显著帮助部分高成本配置，但并非处处更快。在 arXiv 的 $\alpha = 200$、批大小 1,000、八组配置上，同一次配对剖析中，合并成本约由 2,629 降至 471 μs，扫描则由 7,968 降至 5,513 μs。结果支持减少过量中间工作，而不是“扫描分组越多越好”。

另一种未采用的方案是分配更小的静态局部配额。先导工作负载的分布测量显示，第一候选组包含 46–81% 的全局保留候选。等额配额会经常需要正确性回退；即使校准不等额配额，主导分组仍处于关键路径。这些观测促成了精确共享界设计，但它们只是这些工作负载上的先导测量，不能证明所有可能的配额策略都更差。

表 7.4: 精确 FP32 探针选择改动的 v69/v65 端到端吞吐量比。每个工作负载—批大小比较中，每个二进制程序共计时十轮，采用交替运行。大于一表示 v69 更快。

数据集	批大小 8	32	256	1,000
arXiv	1.074	1.063	1.035	1.034
Vogue	1.109	1.103	1.052	1.041
OpenAI-3072	1.167	1.117	1.055	1.050

7.4.3 精确探针选择与流水线一致性的边界

当筛选变快后，IVF 探针选择在小批次时间中的占比会提高。基数探针原语（v69）改变了选取最小质心评分的实现，同时保留 FP32 评分。其合成检查覆盖 1,792 个用例，失败数为零。在真实 arXiv、Vogue 和 OpenAI-3072 工作负载上，检查发现全部 1,000 条查询的探针标识符相同，主码 top-c 位置与距离也相同。这比“召回率数字没有变化”更具体，因为它直接检查了路由和筛选输出。

表 7.4 显示端到端吞吐量提高至 1.034–1.167×。批大小为 8 时，剖析得到的探针阶段在 arXiv 上从 56.5 降至 22.6 μs，在 Vogue 上从 52.7 降至 14.7 μs，在 OpenAI-3072 上从 77.8 降至 16.7 μs。大批次下，未修改阶段占整体耗时比例更高，因此整体收益较小符合预期。这些是各自实验组内部的比值，不能把它们与前面的 v62 或精排收益相乘，仿佛所有测量都共享同一个基线。

Vogue 区分了探针原语与跨进程流水线可复现性。在一组、批大小 8 的测试中，两种二进制的探针/主码 dump 对全部 1,000 条查询逐字节相同，但最终召回率为 0.9642 与 0.9644。同一个基线二进制也从一致性运行中的 0.9642，变成之后两次计时运行中的 0.9644。因此，观察到的变化并不需要探针改动才能发生；在 dump 控制的用例中，它出现在已记录主码选择边界之后。残差表示、输入和评分计算仍是可能来源，根因尚未确定。在各被检进程内部，调度与单线程块输出仍逐位一致。附录 A明确了 dump 的覆盖范围，避免将其当作对每条计时路径的检查。

最后，可选的 v70 FP16 实验属于另一项近似。在 arXiv 的 936/1,000 条、Vogue 的 979/1,000 条、OpenAI-3072 的 998/1,000 条查询上，其探针集合与 FP32 完全相同，平均集合重叠率很高。但高重叠率和很小的召回率变化，并不能使该原语变成精确操作。这些结果不纳入主要 E1 前沿，也不纳入精确探针选择结论。

7.5 留出查询上的预算质量、吞吐量与回本

RQ3 考察相对可信固定预算的质量和摊销成本。因此，比较同时包含沿用的 $\alpha = 100$ 与更强的统一固定对照 $\alpha = 64$。

7.5.1 无标签选择与使用标签的诊断性 oracle

预算实验在每个数据集—探针数组合中固定一个训练好的索引，比较固定预算、抽样校准、全校准池校准、同样本枚举与离线 oracle。六个数据集均测试 128 和 512 探针。偶数下标查询组成 500 条校准池；奇数下标查询组成互不重叠的 500 条留出池，Stella 例外，留出池为 499 条。从校准池中使用固定随机种子抽取 $S \in \{32, 64, 128\}$ 条查询，每种大小重复抽取 64 次；单次抽样内不放回。重复抽样使用的是同一个有限查询池，并不等于对未来工作负载漂移进行 64 次独立采样。

降序预算网格为 $A = (200, 100, 64, 32, 16, 8, 4, 2)$。校准将候选预算的 top-10 标识符集合与 $\alpha_{\max} = 200$ 的参考结果比较。对于样本 $\mathcal{Q}_S$，接受统计量为

$$D_S(\alpha) = \sum_{q \in \mathcal{Q}_S} \left(10 - |\mathcal{R}_\alpha(q) \cap \mathcal{R}_{200}(q)|\right), \qquad D_S(\alpha) \le 1. \tag{7.1}$$

允许的是整个样本合计一个结果槽位不一致，而不是每条查询允许一个，也不是基于 ground truth 的召回率容差。实际规则不接收标签。参考结果本身也是近似结果，因此完全一致也可能复制其错误。

离线 oracle 的作用不同。它查看八个固定预算在留出池上的召回率后，在距最佳实测留出召回率不超过 10^{-3} 的配置中选择最快者。这是诊断性比较标准，不是可直接部署的无标签决策。“全校准池”同样表示在全部 500 条校准查询上应用同一校准搜索，而不是利用留出标签枚举所有预算。这些区分避免将“选到与 oracle 相同预算”误解为质量保证。

7.5.2 预算质量与抽样风险

表 7.5 给出了 $S = 128$ 的结果。十二个单元中，所选预算的中位数有十个与 oracle 预算一致；不一致的两个是 arXiv 的 128 探针配置和 OpenAI-3072 的 512 探针配置。所选预算跨越 4–200，显示出明显的工作负载依赖。然而，预算中位数与平均召回率描述的是抽样总体的不同方面，二者不一定对应某一个配置的性能。此外，表中平均召回率由日志打印的最佳召回率减去打印的平均损失重构，所以保留到四位小数的值是近似值。

一个重要反例是 arXiv。在 128 探针下，即使平均损失仅为 0.0009，仍有 40/64 次抽样的召回率损失超过 10^{-3}；512 探针下有 32/64 次超标。BGE 的 512 探针配置有 24/64 次超标，95 分位损失为 0.0014。Stella 超标次数较少，但两个探针数下的 95 分位损失均为 0.0030。因此，平均质量、超标频率与尾部损失幅度必须一起看。OpenAI-3072 在这些抽样中未观察到超标，这支持其在当前查询池上的有利表现，却不是对未见查询的零风险保证。

表 7.5: $S = 128$ 时的留出质量。规则的 α 是 64 次抽样的中位数，oracle 的 α 使用留出标签。$\overline{R}_{\text{rule}}$ 是重构的平均召回率。“超标”统计相对最佳被测留出召回率损失 $> 10^{-3}$ 的抽样次数；L_{95} 是日志采用较低次序统计量得到的 95 分位损失估计。R_{100} 是固定预算召回率。

数据集	探针数	α: 规则/oracle	R_{100}	$\overline{R}_{\text{rule}}$	超标	L_{95}
Vogue	128	64 / 64	0.9680	0.9677	5/64	0.0022
arXiv	128	100 / 200	0.9748	0.9751	40/64	0.0024
OpenAI-1536	128	8 / 8	0.9280	0.9279	4/64	0.0000
OpenAI-3072	128	4 / 4	0.9252	0.9252	0/64	0.0000
BGE	128	32 / 32	0.9198	0.9190	1/64	0.0008
Stella	128	16 / 16	0.9892	0.9884	6/64	0.0030
Vogue	512	64 / 64	0.9916	0.9915	2/64	0.0002
arXiv	512	200 / 200	0.9878	0.9885	32/64	0.0016
OpenAI-1536	512	8 / 8	0.9740	0.9738	4/64	0.0000
OpenAI-3072	512	4 / 8	0.9704	0.9702	0/64	0.0002
BGE	512	64 / 64	0.9670	0.9661	24/64	0.0014
Stella	512	16 / 16	0.9954	0.9945	7/64	0.0030

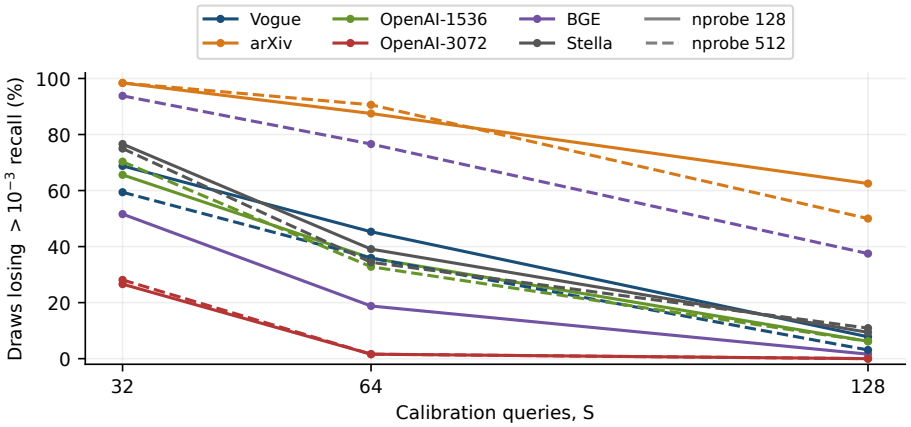


图 7.4: 不同样本大小下的留出风险。纵轴为相对最佳被测预算，留出召回率损失超过 10^{-3} 的校准抽样百分比。由于式 (7.1) 始终保持阈值一，增加 S 也收紧了允许的槽位不一致比例。

对于数量较少的抽样，分位数约定也很重要。测试程序报告排序损失中从零起算的 $\lfloor 0.95(64-1) \rfloor = 59$ 项，而不是在次序统计量之间插值。因此，可能同时存在四个非零尾部损失，而打印的 $L_{95} = 0$，OpenAI-1536 就是如此。表格保留日志统计量与准确超标次数；该列为零不等于所有抽样都没有损失。

图 7.4 显示了样本大小的影响。更大的样本通常使选择更保守，但同时收紧了归一化允许量 $1/(10S)$。在全部 2,304 次抽样（$12 \times 3 \times 64$）中，同样本枚举与二分搜索一致，而打印日志显示枚举耗时为其 1.75–2.01 倍。

被测默认路径执行精确主码 top-c，最终 top-k 遍历全部 c 个精排候选，并按 ID 打破并列。有损的 `K_LOCAL=4` 保留路径是设置 `JHQ_EXACT_TOPCK=0` 才启用的

表 7.6: 十二个留出单元上的固定预算对照。达标表示按打印精度，距最佳被测召回率不超过 10^{-3}。最大召回率损失使用全部单元，QPS 差额只统计该预算达标的单元。横线表示没有达标单元。

α	达标单元	最大召回率损失	达标单元最大 QPS 差额
2	0/12	0.3100	–
4	2/12	0.1774	2.3%
8	4/12	0.0930	2.1%
16	6/12	0.0408	4.4%
32	7/12	0.0170	9.1%
64	10/12	0.0050	26.6%
100	10/12	0.0016	40.9%
200	12/12	0.0000	57.1%

替代路径，并不是本预算实验单调性限制的原因。在评分固定、采用共同全序并列规则时，精确主码选择得到嵌套候选集，因此抽样一致性准则满足单调性。剩余限制是一块路径在截断并列处只比较距离，以及批大小会决定使用单线程块还是按完整键排序的分组扫描。因此，小校准批次和留出查询批次可能采用不同执行路径。枚举一致验证了这个网格上的抽样决策，并不保证留出召回率，也不保证跨路径无条件嵌套。

固定预算也并不完美。表 7.6 按诊断性质量目标评估了全部八个固定选项。固定 $\alpha = 100$ 在十二个单元中有十个达标；$\alpha = 200$ 全部达标，但可能牺牲较多吞吐量。全校准池校准也不等于 oracle：在 Vogue 两个探针数下，它均选择 100，而 oracle 选择 64。即使增加校准样本，基于输出一致性的选择与“质量约束下最快”的选择仍解决不同问题。全校准池耗时为 9.5–60.3 ms。在这十二个单元中，$\alpha = 200$ 达到最高实测召回率，因此作为高预算覆盖对照；12/12 达标不是独立的质量保证。网格中的吞吐量也并非严格单调：例如，OpenAI-3072 在 128 探针下，$\alpha = 2$ 为 128.7 kQPS，$\alpha = 4$ 为 145.0 kQPS。三次计时无法区分时间波动与配置相关执行成本，所以本章不为该倒置认定机制。

7.5.3 估计吞吐量与校准经济性

与 E1 不同，这组实验在预热后，对每个固定配置进行三次搜索计时。规则报告的 QPS 是 64 次抽样所选配置的已有实测 QPS 的算术平均，而不是对一个重复执行“校准—搜索”的服务进行新计时。当 $S = 128$ 时，这一所选配置吞吐量估计在十个单元中优于固定 $\alpha = 100$，比值为 1.028–1.691×。arXiv 的比值则为 0.908 和 0.917：平均召回率提高，但吞吐量降低。这两个单元无法通过吞吐量收益回本。

更强的 $\alpha = 64$ 对照缩小了整体收益：所选配置的平均 QPS 在 9/12 个单元中胜出，几何平均比值为 1.054×，范围为 0.821–1.367×。Vogue 几乎不变；BGE

表 7.7: 校准经济性，$S = 128$。速率单位为 kQPS，规则速率为所选配置平均 QPS。比值分别相对固定 100 与固定 64。最后一列回本相对固定 100，使用 500 条查询，Stella 为 499；“不回本”表示没有吞吐量回本。

数据集	探针数	Q_{100}	$\overline{Q}_{\text{rule}}$	相对 100	相对 64	校准 ms	回本批数
Vogue	128	202.9	224.4	1.106	1.004	4.0	17.0
arXiv	128	207.9	188.8	0.908	0.821	4.2	不回本
OpenAI-1536	128	167.5	268.5	1.603	1.367	3.9	3.5
OpenAI-3072	128	85.7	145.0	1.691	1.230	5.7	2.4
BGE	128	116.2	131.2	1.130	1.051	6.5	13.2
Stella	128	95.7	110.3	1.153	1.087	7.5	10.8
Vogue	512	100.1	102.9	1.028	0.996	7.5	55.4
arXiv	512	83.1	76.2	0.917	0.870	9.7	不回本
OpenAI-1536	512	86.6	115.0	1.327	1.268	8.8	6.2
OpenAI-3072	512	42.8	52.1	1.219	1.052	13.1	6.2
BGE	512	49.2	51.1	1.039	1.006	15.8	41.2
Stella	512	34.6	36.0	1.041	1.028	21.3	37.1

的 512 探针配置只提高 0.6%；OpenAI 的收益范围为 1.052–1.367×。固定 64 和固定 100 都在 10/12 个单元达标，失败的都是两个 arXiv 单元。校准的价值是适应不同精排需求，而不是始终击败已经合理的固定预算。

arXiv 的超标是相对最佳被测留出召回率，不是相对默认配置。固定 100 在 128 和 512 探针下本来分别损失 0.0012 和 0.0016，而抽样平均损失在两个单元均为 0.0009。因此，即使部分抽样超过相对 oracle 的目标，平均质量仍比默认配置更好。

日志中的校准成本为 3.9–21.3 ms。它包含参考搜索与输出一致性检查，以及测量搜索调用内包含的相关设置，但不包含将抽样向量收集到输入缓冲区的操作。对于相同大小的后续批次，定义

$$B^* = \frac{T_{\text{cal}}}{T_{100} - T_{\text{rule}}}, \qquad T_{\text{rule}} < T_{100}. \tag{7.2}$$

使用 $T_{\text{rule}} \approx n/\overline{Q}_{\text{rule}}$ 得到表 7.7中的估计：有利单元相对固定 100 约需 2.4–55.4 个留出池大小的后续批次。相对固定 64，九个有利单元约需 5.7–404.3 批，Vogue 的 512 探针配置没有吞吐量回本。图 7.5对两个对照都使用实际留出查询数。

这种计算偏乐观。一般而言，$1/\mathbb{E}[Q] \le \mathbb{E}[1/Q]$，所以当不同校准抽样选择了速率不同的配置时，对平均 QPS 取倒数会低估平均执行时间。样本收集、未来查询漂移与重新校准开销也会影响摊销。对于长期稳定的工作负载，复用策略可能带来收益；短任务或变化的查询总体则未必如此。当前实验没有测量漂移检测，也没有测量生产环境中的重新校准计划。

作为历史比较，较早的标量精排实现（v57）在 10/12 个单元中报告相对固定 100 的 1.054–2.773× 收益，校准耗时为 5.2–30.8 ms。较新的 lane 精排流水线收益

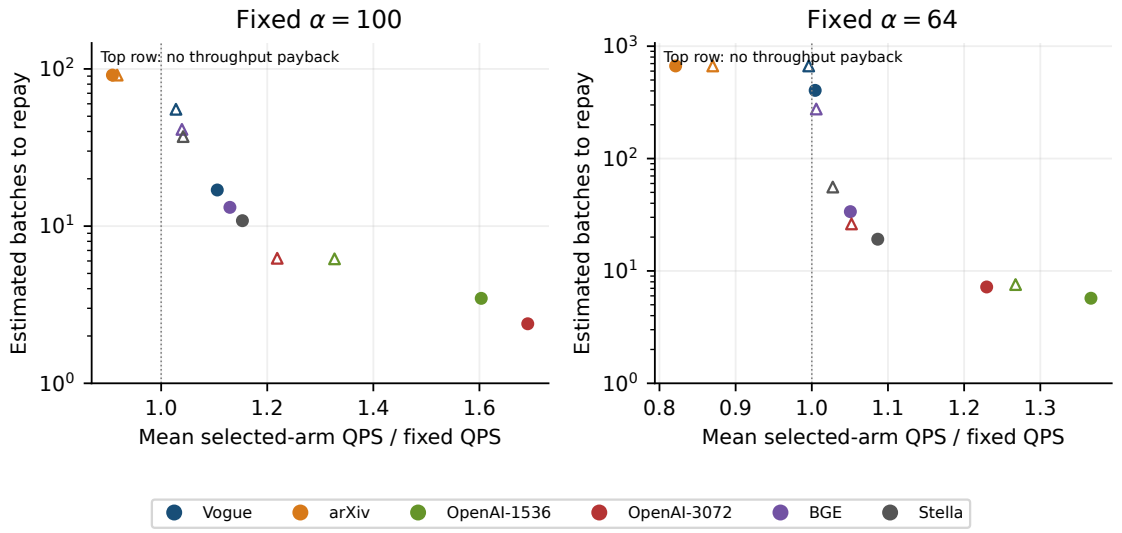


图 7.5: 相对固定预算 100 和 64 的校准回本估计。实心圆表示 128 探针，空心三角表示 512 探针；顶行点没有吞吐量回本。估计对所选配置平均 QPS 取倒数，使用 500 条查询，Stella 为 499。

更小，与可节省精排时间减少的解释相容。两次运行共享预算协议和网格，但跨实验比较并未隔离证明精排是变化的唯一原因。

7.6 批大小、扫描组调度与工作负载偏斜

本节回答 RQ2 的性能区间问题：何时额外扫描组提供有价值的并行度，何时额外工作成为主导。

7.6.1 并行度与重复选择工作的权衡

扫描实验揭示了两种执行区间。每条查询使用一个线程块时，批大小 8 只在具有 170 个 SM 的设备上提供八个线程块。划分候选能增加可用线程块并行度。在 v62 实验中，批大小 8 使用八组后，端到端吞吐量为原来的 2.06–2.76×。剖析给出了阶段解释：arXiv 扫描约从 849 降至 216 μs，Vogue 从 1,059 降至 246 μs，OpenAI-3072 从 728 降至 155 μs。额外合并分别约需 16.7、9.1 和 3.7 μs。即使支付合并成本，净节省仍很大。

批大小为 1,000 时，独立查询已足以提供较多线程块。增加分组会重复搬入查找表、接纳局部候选并排序，还产生额外合并流量。对于 arXiv 的 $\alpha = 200$ 配置，不共享界的 v62 路径可能降至单线程块基线的约 0.46×。全批次剖析显示，$\alpha = 100$ 的扫描从 2,922 增至 6,079 μs，$\alpha = 200$ 从 3,530 增至 7,652 μs，后者还增加了 2,624 μs 合并时间。这一反向结果对设计至关重要：并行拆分能够解决并发

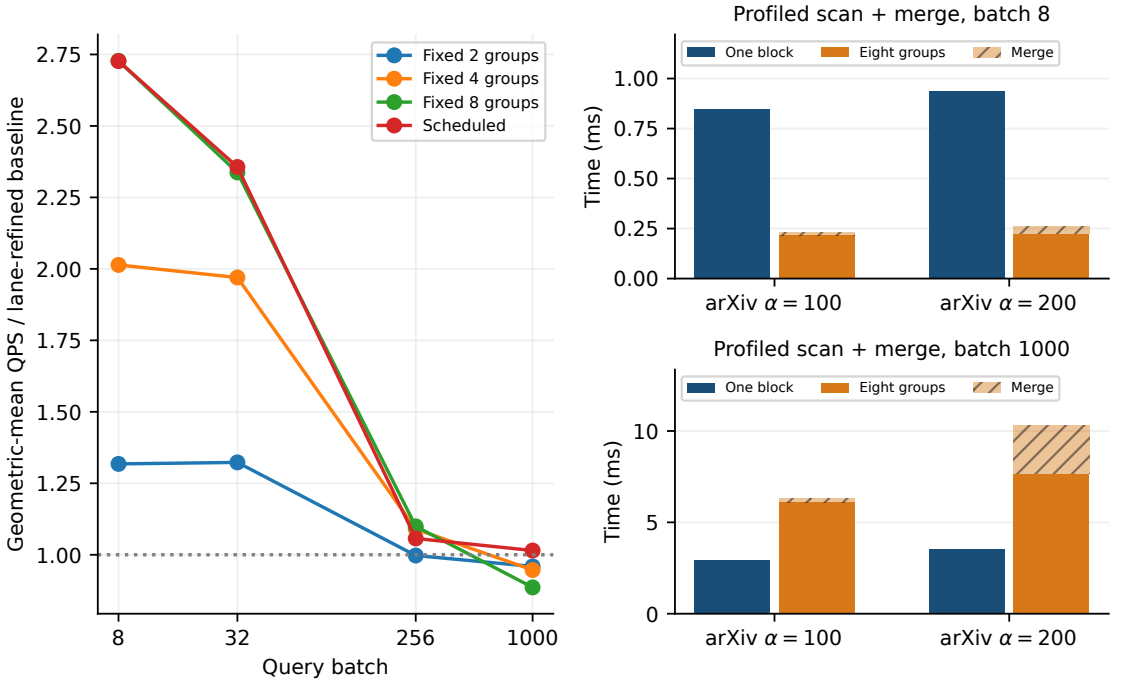


图 7.6: 执行区间。左：共享界固定组数与调度执行，相对 lane 精排单线程块基线的吞吐量比，较小批次有 13 个单元，批大小 1,000 有 11 个。右：arXiv 预算 100 和 200 在批大小 8 与 1,000 下的配对扫描、合并剖析；斜线表示合并时间。右侧来自独立界扫描实验，用于解释一项成本，不代表调度器全部收益。

不足，但当并发已经充足时，它引入的额外工作可能成为主导。共享界减少了这些额外工作，却没有消除执行区间的变化。

7.6.2 在更多配置上评估预先固定的规则

集成批调度器（v65）根据批大小、线程块大小以及工作负载偏斜估计选择扫描组数。填充规则将潜在线程常驻数量与设备线程容量的固定 0.6 比例关联；偏斜规则则在每条查询候选数的变异系数至少为 0.3 时，要求至少使用四组。最终取两条规则建议的较大组数；一组情况映射到原始单线程块扫描。偏斜统计只估计一次，使用 512 个数据库向量作为伪查询，不使用 ground-truth 标签或被测查询池。规则常数在四个工作负载配置上确定，之后才测量额外配置。

表 7.8 从原始计时数组重新计算结果。批大小为 8、32、256 和 1,000 时，调度收益的几何平均分别为 2.727×、2.356×、1.057× 和 1.015×。前三种批大小有十三个配置，其中包含额外的 arXiv 高预算配置；批大小 1,000 只有十一个，因为 Stella 的 999 条查询池无法组成一个完整的 1,000 查询批次。对应的留出配置子集分别含九个和七个配置。因此，将这些均值连接起来的图总结的是运行区间，而不是完全平衡的重复测量总体。

表 7.8: 集成执行相对 v61 L16 的收益。每项先计算单元内五轮配对加速比的中位数，再跨单元取几何平均。“留出配置”排除了设置规则时使用的四个配置，指的是未用于设定规则的配置，不是预算实验中的留出查询划分。

批大小	单元数	$g_s = 2$	$g_s = 4$	$g_s = 8$	调度	留出配置
8	13	1.318	2.014	2.728	2.727	2.930
32	13	1.323	1.970	2.337	2.356	2.560
256	13	0.997	1.090	1.099	1.057	1.041
1000	11	0.959	0.947	0.886	1.015	1.012

固定八组在批大小 8 时的均值接近调度结果，但在批大小 1,000 时降至 0.886×。调度大体保留了小批次收益，并避免上述大批次损失。不过，它仍是启发式规则，不是 oracle。批大小 256 时，在一个 BGE 配置上，“最佳固定吞吐量 / 调度吞吐量”的最差比值为 1.217。候选数 CV 无法描述列表遍历、选择或合并的全部成本。这一失败表明所选特征存在局限，并不说明执行调度没有必要。

Vogue 说明仅看批大小是不够的。其候选数 CV 实测约为 0.51，而原始其他工作负载约为 0.08–0.11；其最重查询的候选数约为均值的 2.2 倍。划分候选能缩短慢查询，而该查询原本可能决定整个同步批次的完成时间。这解释了为何 Vogue 在较大批次下仍可能受益于多组执行，不过这些 CV 观测仍属于工作负载证据，并不是对偏斜作用的普遍因果隔离。

最后，正确性检查独立于计时。在 250 个变体/配置比较、234,320 次逐查询比较中，集成实验未发现非并列主码选择差异，也未发现最终标识符集合差异。在并列情况下，最终顺序可能不同，因而不能将整个实验描述为无条件逐位一致。吞吐量结果针对顺序同步提交，不足以建立开放到达过程下的尾延迟表现。

7.7 冷构建、常驻显存与已验证规模

通过冷构建、常驻显存与已完成的大数据集搜索评估 RQ1；E1 显存比较也为 RQ4 提供依据。

7.7.1 冷构建与生命周期边界

主要搜索扫描复用了已训练状态缓存。加载该状态并添加向量不属于冷构建。独立日志明确在训练状态缓存为空时训练，然后编码并添加数据库；表 7.9给出这些记录中的训练与添加时间。合计耗时从 OpenAI-1536 的约 1.01 s 到 Stella 的 10.40 s。Stella 的训练为 3.74 s、添加为 6.67 s，也说明不能仅凭另一个阶段或数据集规模可靠推断某个阶段的成本。

表 7.9: JHQ 冷构建与单独测量的常驻显存。训练和添加来自每个数据集的一次已记录冷构建；显存来自 v71 两组、128 探针、$\alpha = 100$ 的运行，包含上下文与工作区。“残差占比”表示模型计算的残差码占实测已用总量的比例，而不是每次查询读取的比例。

数据集	训练 s	添加 s	合计 s	已用 GiB	原始 GiB	残差占比
Vogue	1.06	0.27	1.33	1.47	2.67	45.4%
arXiv	0.59	0.52	1.10	2.58	6.45	62.5%
OpenAI-1536	0.64	0.38	1.01	2.39	5.72	59.7%
OpenAI-3072	1.34	0.65	1.99	4.14	11.43	69.0%
BGE	3.75	3.67	7.42	11.93	38.50	80.7%
Stella	3.74	6.67	10.40	20.44	67.81	82.9%

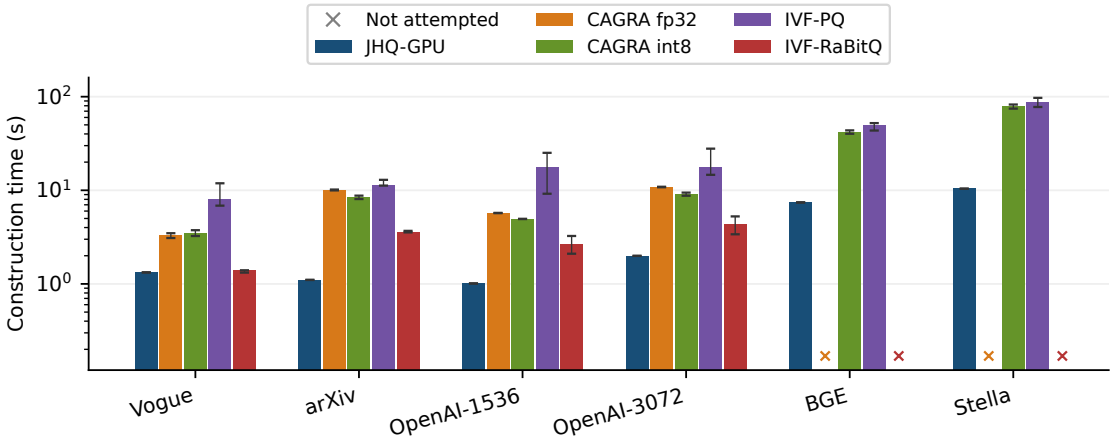


图 7.7: 包括 CAGRA int8 的已记录构建成本。JHQ 为一次冷训练加添加记录；其他柱形为成功测量的索引配置之间的中位数，误差线表示最小值至最大值。Int8 包含数据集量化；RaBitQ 不包含所需的序列化往返。大数据集缺失的柱形表示未尝试的配置，不代表实测构建失败。纵轴为对数坐标。

这里的“冷”指已学习索引状态。输入加载器可复用内存映射 FP32 数据；报告的训练加添加区间不包含从存储设备读取原始导出文件。调度器的伪查询校准在添加之后进行，也不计入总量。完整部署生命周期还应考虑数据准备、某些接口未计入的传输、校准、序列化和后续查询时间。这些记录是在明确边界内有价值的构建测量，不是完整端到端数据摄取服务的成本。

比较图同时给出了其他构建配置与这些冷构建记录。每种方法的范围不是等价重复构建的置信区间。训练样本、索引参数与表示不同，而且每个数据集只有一次 JHQ 冷构建记录。对于 RaBitQ，记录中的构建函数与序列化/反序列化分开计时，但基准必须完成该往返后才能搜索。只有将柱形明确标为构建函数时间时，才可以不计这一步；它并不表示直到第一次可用搜索的全部时间。

在 Stella 上，CAGRA int8 成功的隔离构建按图出度不同，包含数据集量化后约为 74.55 和 82.45 s；IVF-PQ 三个配置的实测构建中位数为 86.59 s。在报告边界内，JHQ 的 10.40 s 明显更短，但不能据此证明它在所有条件下构建最快，也

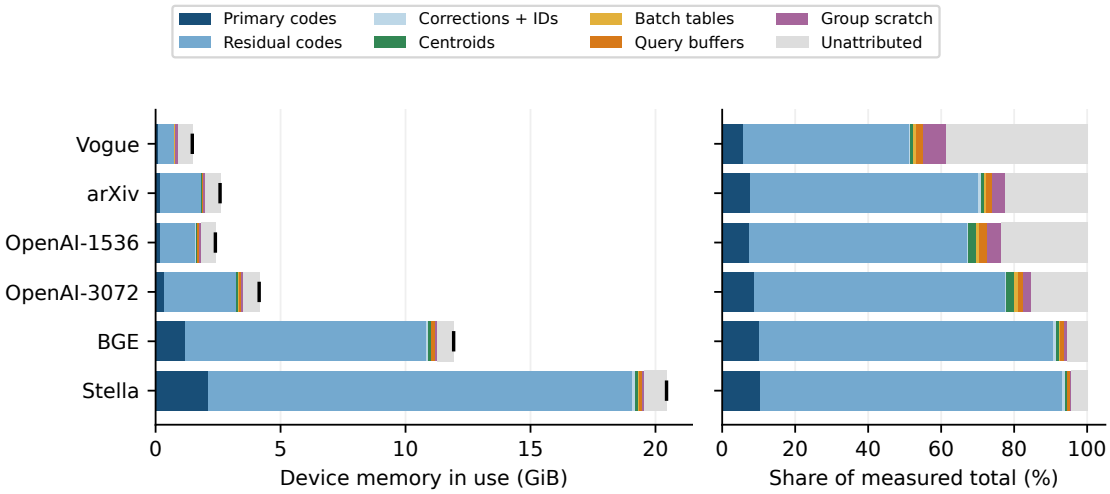


图 7.8: 两组查找表、128 探针、$\alpha = 100$ 下的常驻显存分解。组件大小由模型计算，标记为实测已用总量，斜线区域为未归因部分。总量包含上下文与工作区，未测量构建峰值。

不能证明后续召回率相同。构建速度、可搜索质量与常驻成本是部署时需要分别考虑的维度。

7.7.2 常驻分配与压缩发挥作用的位置

图 7.8 分解常驻显存。标记表示记录时设备已用显存，而不是纯索引分配。堆叠部分用模型计算主码、残差码、修正因子与标识符、质心、每批查找表、候选缓冲区和列表分组临时区。剩余实测分配明确保留为未归因部分；它可能包含运行时上下文、分配器影响以及模型遗漏的分配。将剩余部分全部归给残差码，会高估表示成本。

在大数据集上，残差码是主要占用：BGE 的已用总量为 11.93 GiB，其中约 80.7% 为残差码；Stella 为 20.44 GiB，其中约 82.9% 为残差码。仅对应的原始 FP32 向量数组就分别需要 38.50 和 67.81 GiB。因此，JHQ 通过保留压缩层次表示，使这些已测试的数据能够在单卡常驻搜索。降低 α 会减少每条查询访问、评分的残差码，但不会释放已经存储的残差数组。同样，笛卡尔 LUT 因子化减少的是查询相关查找表存储，不是数据库残差码的精度。

工作区同样重要。在这些 $\alpha = 100$ 测量中，报告的列表分组临时区约为 93.8 MiB，即使实际执行选择单线程块路径，也为批容量 1,024 和最多八组预留。因此，实际组数变小，不意味着分配按比例缩小。另一方面，这些常驻样本没有捕获临时构建峰值。基线测试程序也在测量索引显存之后才初始化部分搜索内存池，因此，不能把构建后的索引增量当作各方法完整进程峰值显存预算。

E1 测量提供了表 7.10中的独立跨系统视图。从 JHQ 进程总量减去基线测得的 507.2 MiB 上下文后，约为 CAGRA fp32 索引分配的 0.29–0.39。JHQ 数字包

表 7.10: E1 分配范围。JHQ 为进程总量减去 507.2 MiB 上下文估计，包含查询工作区；CAGRA fp32 为构建时测量的索引分配。比值范围组合每个数据集的极值。

数据集	JHQ 净进程 GiB	CAGRA fp32 索引 GiB	比值范围
Vogue	0.90–1.01	2.89–2.91	0.31–0.35
arXiv	2.01–2.19	6.84–6.98	0.29–0.32
OpenAI-1536	1.79–2.33	5.95–5.97	0.30–0.39
OpenAI-3072	3.54–3.82	11.67–11.68	0.30–0.33

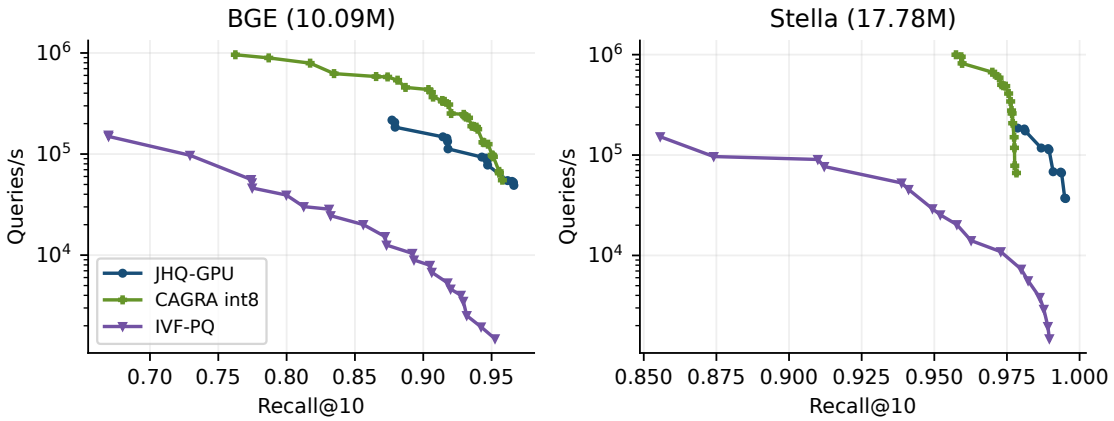


图 7.9: 大数据集上的主机到主机前沿，包含 CAGRA int8 成功的隔离重跑。BGE 使用 1,000 条查询，Stella 使用 999 条。曲线端点是已测试网格的终点，不是方法的内在召回率上限。

含工作区，CAGRA 数字则是构建时索引增量。这些是参数扫描范围，不是显存匹配的同召回率运行点，也不是相同峰值预算的比较。

7.7.3 大数据集可运行性与竞争区间

BGE 含 10,091,524 个向量，Stella 含 17,776,615 个向量，均为 1,024 维、32,768 个 IVF 列表。Stella 查询采样自语料库，因此结果描述的是该基准，而不是独立收集的外部查询分布。图 7.9 包含成功的 CAGRA int8 和 IVF-PQ 测量。对于计算存储需求超过可用设备容量的全精度 CAGRA、RaBitQ 配置和 FP32 暴力搜索，大规模比较没有尝试运行。这些缺项是预先进行的可运行性排除，不是观察到的显存耗尽错误或实测召回率失败。

成功的隔离重跑显示，CAGRA int8 能在被测配置下装入 Stella。在 BGE 的共同采样目标 0.88/0.90/0.92/0.94/0.95 上，int8 约比 JHQ 快 1.35–2.96×；在 int8 的最高实测召回率 0.9579 处，JHQ 插值 QPS 是 int8 的 1.086×。因此，优势在高召回率末端附近反转。BGE/Stella 上，int8 的最高被测召回率为 0.9579/0.9783，JHQ 为 0.9663/0.9952，PQ 为 0.9525/0.9896。

Stella 上一个具体高端比较为：JHQ 在召回率 0.978679 时达到 185.4 kQPS，int8 在 0.978278 时为 66.3 kQPS，速率比为 2.80×，两者召回率略有不同。这是一对明确的实测点，不是统一的匹配召回率优势：JHQ 很多更高召回率区间没有 int8 覆盖，而且其最高召回率点也以吞吐量换取质量。附录 A记录修正运行的排除规则与测试程序诊断。

结果证明的是已测试规模和有价值的高召回率区间，而不是任意规模的可扩展性。最大已完成搜索是在单 GPU 上处理 17.78 百万个向量；本章没有多 GPU 实验，没有保持数据分布不变的扩展曲线，也没有接近显卡容量时的构建峰值证明。Stella 搜索使用实际的 999 条查询池。

7.8 数值稳健性、异常与召回率归因

稳健性检查界定 RQ2 的精确性声明、RQ3 的风险评估与 RQ4 中较小的跨系统差距。

7.8.1 质量分辨率与重复执行

当 $k = 10$、查询数为 1,000 时，多找回一个邻居会使总体 Recall@10 改变 10^{-4}；500 条留出查询时，分辨率为 2×10^{-4}。Stella 的 499 查询划分略粗。这种离散分辨率不是不确定性区间，但有助于理解前面的小差异。第四位小数的变化可能只对应一两个结果槽位，而小的平均损失仍可能掩盖若干答案变化较大的查询。

重复计时调用也必须与重复索引构建分开。E1 解析器记录了 369 个标识符确定性标记为 false 的基线运行点：170 个 RaBitQ、80 个 IVF-PQ、68 个 CAGRA int8、50 个 CAGRA fp32，以及一个暴力搜索点。该标记可能包括并列结果的顺序变化，本身不意味着标识符集合变化或输出无效。没有测量点仅因这一标记而被丢弃。原始日志保留首次/末次召回率与集合、顺序诊断。JHQ 没有填写同样的基线标记，因此不能把它缺失该标记当作统一的跨方法确定性测试。较早 E1 报告的 401 使用的是 369 个 A 重复调用标记与 367 个 B 相对 A 不匹配标记的并集；其中 32 个点仅增加后一种标记。B 与 A 不一致是跨协议相等性检查，不等于重复调用 B 不确定。

五轮吞吐量摘要保留了固定构建内部的变化。最小值和最大值是描述性范围，不是置信区间，也不包括训练差异。因此，对两个 LUT 接近平手的例子、或 arXiv 与 RaBitQ 在召回率 0.97 处的小幅差距，不应赋予太大的解释权重。与对应阶段时间减少相匹配、幅度较大且能重复的收益，比单独一处微小前沿移动更能支持执行优化结论。

在各数据集 RaBitQ 配置间，“五轮最大值减最小值 / QPS 中位数” 的中位数分别为：Vogue 3.79%、arXiv 3.34%、OpenAI-1536 2.87%、OpenAI-3072 5.25%。这些描述性波动与 Vogue 小幅领先的量级接近，而不是匹配比值的置信区间。原始调用轨迹也包含长调用，在原本较快的配置中偶有 30–60 ms 观测。敏感性检验对 1,000 条查询的一次调用，按 $10^6/\,\mathrm{median(call\ latency\ in\ ms)}$ 重算 QPS，以减小长调用的影响。召回率 0.97 处，得到的 JHQ/RaBitQ 比值为 1.063、0.995、1.118 和 1.100。三个有利方向与 arXiv 接近平手的格局保持，因此结果不只是长调用造成的。

7.8.2 精确性必须限定到具体阶段

几项不同声明必须区分。笛卡尔因子化在代数上保持已表示评分；打包加载保持逻辑编码值；采用完整键的局部保留和合并恢复已访问候选中的精确主码 top-c；共享界保持该选择目标；FP32 探针原语通过了合成与真实输入的等价检查。以上任何一项，都不等于在原始向量空间找到了精确最近邻。

lane 归约会改变舍入；主码截断边界的并列可能改变被精排的候选；最终选择对全部保留的精排评分进行精确选择，并按 ID 打破并列；相对原空间邻居的剩余误差可以来自此前阶段或已表示评分。归档中的 Vogue 跨版本差异说明，完整流水线的一致性需要独立检查。类似地，FP16 路由实验表明，几乎不变的平均召回率也不足以证明探针精确等价。因此，主要正确性结论都限定到实际测试过的阶段与输入覆盖。

召回率损失可能来自路由、主码筛选、残差评分或最终选择。增加探针扩大路由覆盖；增加 α 使更多主码候选进入残差评分；二者都不能恢复后续所有近似阶段丢失的信息。预算扫描固定索引和探针配置，直接测量改变保留候选数量的影响，却没有把每个遗漏邻居分解为唯一原因。完整逐查询归因需要在路由、主码保留和精排后分别检查真实邻居覆盖，并对同一候选集使用精确最终选择作为参考。本章所用实验尚未建立这种完整归因，因此不为未测量的损失来源分配百分比。

7.8.3 研究发现的适用范围

表 7.11 收束 RQ1–RQ4 的证据链。结果支持少量面向表示的执行模式与工作负载相关精排工作量，并分别提供正确性、速度和质量证据。

结论适用于平方 L2 的 top-10 搜索、记录查询池与被测 RTX 5090 环境。跨设备行为、变化查询、在线更新与开放到达服务延迟尚未测试。这些边界与正面结果共同呈现，而不是取代正面发现。

表 7.11: 本章证据支持的四个研究问题答案。

研究问题	证据	答案	边界
RQ1	冷构建、常驻组件、大规模搜索	单卡 17.78M 向量；20.44 GiB；训练加添加 10.40 s	单次冷记录；未测构建峰值或多 GPU 扩展
RQ2	LUT、精排、扫描、共享界、探针配对测试	被测主码选择精确；批大小 8 调度为 2.727×	并列与数值路径；全批次收益小得多
RQ3	互斥查询划分、固定 64/100、oracle、枚举	平均 QPS 相对 64 胜 9/12，相对 100 胜 10/12	校准开销；最多 40/64 次相对 oracle 损失超标；未测漂移
RQ4	六方法 E1 扫描、匹配召回率、显存	0.97 处三个领先，arXiv 接近平手；净分配低于 fp32 CAGRA	int8 在低召回率领先；端点和计时不确定性

第 8 章

Discussion, Limitations and Future Research

待撰写： Synthesize supported findings, analyse two relevant negative designs, and state internal and external validity limits.

8.1 Three Cross-Layer Insights

待撰写： Develop the supported links between representation and execution, parallelism and repeated work, and execution cost and the value of budget reduction.

8.2 Two Deep Negative-Result Case Studies

待撰写： For two case studies, state hypothesis, alternatives, test conditions, observations, and stopping decision. Label isolated GPU, offline, estimated, and untested evidence.

8.3 Threats to Internal and External Validity

待撰写： Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

8.4 Original Testable Research Directions

待撰写： Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

8.5 Conditional Deployment Recommendation

待撰写： Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

第 9 章

Conclusion

待撰写： Answer the research questions using only results already established. Do not introduce new measurements or contributions.

9.1 Answers to the Four Research Questions

待撰写： Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

9.2 Closing Principle and Outlook

待撰写： Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

参考文献

[1] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. Cache locality is not enough: High-performance nearest neighbor search with product quantization fast scan. *Proc. VLDB Endow.*, 9(4):288–299, 2015. doi: 10.14778/2856318.2856324.

[2] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. Quicker ADC: Unlocking the hidden potential of product quantization with SIMD. *IEEE Trans. Pattern Anal. Mach. Intell.*, 43(5):1666–1677, 2021. doi: 10.1109/TPAMI.2019.2952606.

[3] Oded Green, Robert McColl, and David A. Bader. GPU Merge Path: A GPU merging algorithm. In *Proceedings of the 26th ACM International Conference on Supercomputing*, pages 331–340, 2012. doi: 10.1145/2304576.2304621. URL `https://davidbader.net/publication/2012-gm-ba/2012-gm-ba.pdf`.

[4] Jiaxin Han, Mengxuan Zhang, and Goce Trajcevski. JHQ: Johnson-Lindenstrauss enhanced hierarchical quantization for high-dimensional approximate nearest neighbor search. *Proc. VLDB Endow.*, 19(7):1530–1543, 2026. doi: 10.14778/3801059.3801067.

[5] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Trans. Pattern Anal. Mach. Intell.*, 33(1): 117–128, 2011. doi: 10.1109/TPAMI.2010.57.

[6] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs, 2017. URL `https://arxiv.org/abs/1702.08734`.

[7] NVIDIA. *CUDA C++ Programming Guide, Release 13.0.1*. NVIDIA, 2025. URL `https://docs.nvidia.com/cuda/archive/13.0.1/cuda-c-programming-guide/index.html`. Sections 10.14.1.4 and 18.5.3.3; accessed 27 October 2026.

[8] Hiroyuki Ootomo, Akira Naruse, Corey Nolet, Ray Wang, Tamas Feher, and Yong Wang. CAGRA: Highly parallel graph construction and approximate

nearest neighbor search for GPUs. In *IEEE ICDE*, 2024. doi: 10.1109/ICDE60146.2024.00323.

[9] Jianyang Shi, Jianyang Gao, Jiaxin Xia, Tamás B. Fehér, and Cheng Long. GPU-native approximate nearest neighbor search with IVF-RaBitQ: Fast index build and search. *Proc. VLDB Endow.*, 19(11):3454–3467, 2026. doi: 10.14778/3836663.3836701.

第 A 章

Reproducibility and Experiment Manifest

A.1 Source and Build Environment

Chapter 7 is an offline analysis of archived measurements in the sibling `JHQ_GPU` repository. The analysis checkout is recorded in `data/ch7/audit.json`; this is distinct from each experimental run's source head. 表 A.1 maps component names to implementations, recorded provenance and executable names. A run-head commit identifies source visible to a run, not a cryptographic identity of its binary. The archives do not supply a complete collection of executable SHA-256 hashes or immutable trained-state hashes. Missing identities are left explicit rather than reconstructed from the current checkout.

The principal environment is RTX 5090, driver 595.71.05 and CUDA 13.0; E1 uses cuVS 26.08.01. Release JHQ targets compute_80 PTX, JIT-compiled on the device. Native-library baselines and C++/Python interface boundaries differ. E1 improves comparability by reusing resources, preallocating outputs, using pinned input staging and timing int8 query quantization. JHQ and baseline output buffers still have different host memory types. These details limit interpretations of small margins.

The adaptive call budget uses a warm-up interval containing the first call. In the Python baseline harness that interval also includes initial result conversion and validation before the decision; it is not a pure steady-state latency measurement. The 274 reduced-call points therefore include a RaBitQ point above 507 kQPS. Actual calls and five round arrays remain in the JSONL records. The default exact-selection setting was also inspected at the budget run's historical source head, not merely in the current checkout: `JHQ_EXACT_TOPCK=0` opts into the old lossy primary-retention implementation. Final selection scans all retained refined scores and breaks ties by database ID.

表 A.1: Component and run provenance. "Report" distinguishes a version stated in an experiment report from a raw run-header commit. The exact E1 build state is not inferred from a baseline's metadata.

Component	Recorded source evidence	Executable or harness	Run/state source
E1 final pipeline	Environment logs include `a31c22d`; extensions have their own metadata	`demo_jhq_v69_batch`; `bench_cuvs.py`; `bench_rabitq_sweep`	E1 `RAW_RESULTS/`; shared trained-state cache
Lane refinement (v61)	Raw header `b37efa4`	`demo_jhq_v61_ab`	`warp_refine/raw/run.log`; `arms6_cache`
Grouped scan (v62)	Report: `862543a`; run script `e0d0518`	`demo_jhq_v62_batch`	`pipeline_v62/raw/`; recorded compiler flags
Shared bound (v63)	Implementation and build flags archived; no run-head claim inferred here	`demo_jhq_v63_batch`	`pipeline_campaign/exp2/v63_raw/`
Batch scheduling (v65)	Raw header `d425ab8`	`demo_jhq_v65_batch`	`exp47/raw/run.log`; `arms6_cache`
Radix probe (v69)	Comparative runs and source archived; binary SHA not recorded	`demo_jhq_v65_batch`, `demo_jhq_v69_batch`	`practical_opt/v69_run.sh`; `v69_results/dumps.sha256`
Budget / table sweep	Raw header `d5e1526`	`demo_jhq_v69_budget_arms`; `demo_jhq_v71_gN_batch`, $N \in \{1, 2, 4, 8\}$	`adc_v69/RAW/`, `RAW_LUT/`; `arms6_cache`
Large int8 rerun	Stella degree-64 metadata `ad433a4`	`bench_cuvs.py`	`adc_v69/RAW_BIG/`; isolated processes
Historical budget	Raw header `bf31b41` (v57 family)	`demo_jhq_budget_arms.cu` harness	`paper_adc2026/data/budget_arms.log`

A.2 Data and Trained States

The six database shapes and actual query counts are stated in Chapter 7. The comparisons use the same exported vectors and squared-L2 ground truth per dataset. The budget split uses even query indices for calibration and odd indices for evaluation; Stella has 500/499 queries. Search sweeps load the recorded `arms6_cache` trained states, whereas cold-build records explicitly start without trained state. The chapter's file ledger hashes analysis inputs, not the full underlying vector files or every cache object; it does not certify unrecorded data or binary identities.

The Vogue probe comparison stores matching SHA-256 hashes for the one-group, batch-8 probe/primary dumps, covering all 1,000 queries. Its baseline identity run reports recall 0.9642; the same baseline binary's two subsequent timing

表 A.2: Chapter 7 result-to-figure mapping. Paths below are relative to `JHQ_GPU`; detailed file identities are in the hash ledger.

Figure	Raw source
7.1	`experiments/comp8800_poster_e1/RAW_RESULTS/`; `analyze.py` frontier and interpolation definitions
7.2	`experiments/adc_v69/RAW_LUT/v71_g*.json`
7.3	`paper_adc2026/data/layout6.log`; `experiments/warp_refine/raw/ab/` and `raw/summary.json`
7.4, 7.5	`experiments/adc_v69/RAW/budget_v69.log`; actual 500/499 query counts
7.6	`experiments/pipeline_campaign/exp47/raw/bench/`; `experiments/pipeline_v62/raw/perf/stages_summary.json`
7.7	E1 and `adc_v69/RAW_BIG/` cold-build logs; baseline index records, with int8 dataset quantization
7.8	`adc_v69/RAW_LUT/v71_g2_*_np128.json`; modelled component sizes in the chapter analysis script
7.9	`experiments/adc_v69/RAW_BIG/` corrected top-level records

runs report 0.9644. The dump controls a specific routing/screening boundary. It does not store a complete cross-process residual-code, query-rotation or score trace and therefore does not identify the numerical root cause downstream.

Two exclusions affect the scale evidence. Initial large-baseline builds retained an earlier index through a closure; successful isolated reruns supersede those attempts, including Stella CAGRA int8. Separately, Stella's 999-query pool causes the full-batch harness to skip batch 1,000. Top-level corrected large-search records use batch 999; superseded skipped-search files supply no throughput points. Cold-build logs are used for train/add time even where their search list is empty.

A.3 Result-to-Figure Mapping

表 A.2 maps the empirical figures to their measurement families. All Chapter 7 empirical plots and numeric tables are generated by `scripts/build_ch7.py`; `data/ch7/sources.csv` gives consumed input paths relative to `JHQ_GPU` and their hashes. Historical v57 and newer budget comparisons retain separate versions and timing protocols. The memory decomposition combines measured totals with an explicit component model; its unaccounted remainder remains visible.

To regenerate the report, run `make ch7-data` and then `make pdf`. The optional Chinese reading copy uses `make pdf-zh`; its table translator reuses numeric data from the English tables. Regeneration is an offline consistency check, not a new GPU measurement. The original vectors, trained states, matching GPU

software environment and executable build would be needed for an independent benchmark rerun.

第 B 章

Supplementary Results and Correctness Checks

B.1 Complete Parameter Grids

待撰写： List method-specific tuning grids, invalid or unavailable settings, and reasons for unattempted configurations.

B.2 Correctness and Numerical Checks

待撰写： Include reference comparisons for encoding, LUT factorisation, primary selection, probe selection, and refinement. Separate numerical, algorithmic, and memory-safety evidence.

B.3 Additional Results and Negative Designs

待撰写： Include secondary results with evidence status: integrated system, isolated GPU measurement, offline replay, analytical estimate, or untested idea.

第 C 章

Contributions and Tool-Use Record

待撰写： Use the form of disclosure required by Canvas. The tables below are a record template, not an attestation that any particular verification was performed. Replace all entries with a truthful account.

C.1 Contribution Record

Work component	Contributor and role	Supporting record
To complete	*To complete*	*To complete*

C.2 Generative-AI Assistance

Tool and date	Purpose and scope	Prompt record and human review
To complete	*To complete*	*To complete*

第 D 章

Template Examples — Draft Only

This appendix demonstrates the available formatting. It is omitted when `\draftthesisfalse` is selected in `config/metadata.tex`.

D.1 Citations and Cross-References

Use `\citet{jhq}` for a narrative citation: Han et al. [4]. Use `\citep{pq,cagra}` for a parenthetical citation [5, 8]. Examples: 图 D.1、表 D.1与式 (D.1). The bibliography entries are copied from the existing ADC source; review their metadata before submission.

D.2 A Source-Native Diagram

D.3 Tables

D.4 Equations and Proofs

$$D(\alpha) = \sum_{q \in Q_s} \left[k - \left| R_\alpha(q) \cap R_{\max}(q) \right| \right]. \tag{D.1}$$

The environments `definition`, `lemma`, `proposition`, and `proof` are available. For example:

定义 D.1 (Strict total ranking key)**.** A ranking key specifies how to order unequal scores and how to break ties.

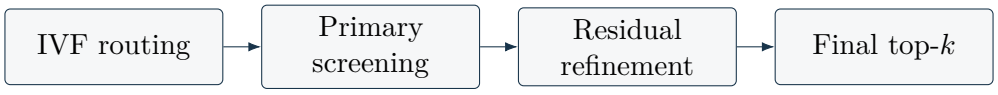


图 D.1: Formatting example: a schematic query path. Replace or extend it with the verified execution boundaries of the implementation.

表 D.1: Example manifest fields. This table contains no experimental results.

Field	What to record
Source	Immutable commit and build configuration
Index	Trained-state identity and input hashes
Timing	Host-to-host or device-resident boundary
Quality	Metric, ground truth, and held-out split
Evidence	Raw log and analysis-script path

表 D.2: Pseudocode layout example.

Line	Operation
1	Read the input and validate the stated preconditions.
2	Compute the intermediate result.
3	Return the output and specify its semantics.

D.5 Pseudocode and Code

The table below gives a package-light pseudocode layout; replace its contents with the actual method and keep correctness premises in the surrounding text.

Listing D.1: Code-listing format example.

```
// Use only short excerpts that help explain a design.
// Keep full implementations in the reproducibility artifact.
void search();
```